Contents

1 C++20 1
- 1.1 format / println . . . 1

2 Basic 1
- 2.1 debug list . . . 1
- 2.2 SVG 視覺化 debug . . . 2
- 2.3 時間複雜度 . . . 2

3 Dark Code 2
- 3.1 IO optimization . . . 2

4 Geometry 2
- 4.1 2D point . . . 2
- 4.2 兩線段交點 . . . 3
- 4.3 兩圓交點 . . . 3
- 4.4 Convex Hull . . . 3
- 4.5 多邊形 / 旋轉卡尺 . . . 3
- 4.6 最近點對 . . . 3
- 4.7 外心 . . . 4
- 4.8 最小覆蓋圓 . . . 4
- 4.9 半平面交 . . . 4
- 4.10 凸包查詢 / 圓切線 / Minkowski . . . 4

5 Flow 5
- 5.1 Dinic . . . 5
- 5.2 min cost flow . . . 5
- 5.3 Gomory-Hu Tree . . . 5

6 Mathematics 6
- 6.1 ax+by=gcd(a,b) . . . 6
- 6.2 exCRT 中國剩餘定理 . . . 6
- 6.3 BSGS 離散對數 . . . 6
- 6.4 原根 . . . 6
- 6.5 GaussElimination . . . 6
- 6.6 Inverse . . . 7
- 6.7 LinearPrime 歐拉篩 . . . 7
- 6.8 Miller Rabin . . . 7
- 6.9 Pollard's rho . . . 7
- 6.10 NTT . . . 7
- 6.11 FFT . . . 8
- 6.12 FWHT . . . 8
- 6.13 Simplex . . . 8
- 6.14 線性基 XOR basis . . . 9
- 6.15 Berlekamp-Massey / 線性遞迴第 k 項 . . . 9
- 6.16 Lagrange 插值 . . . 9
- 6.17 數論基本工具 . . . 10
- 6.18 Mobius . . . 10
- 6.19 數論分塊 (Quotient Blocking) . . . 10
- 6.20 SG . . . 10
- 6.21 歐拉定理 . . . 10
- 6.22 Theorem . . . 11
- 6.23 組合數學公式 . . . 11

7 Graph 11
- 7.1 BCC . . . 11
- 7.2 Prim . . . 11
- 7.3 Bellman Ford . . . 12
- 7.4 Kruskal . . . 12
- 7.5 Dijkstra . . . 12
- 7.6 Hungarian . . . 12
- 7.7 Hopcroft-Karp . . . 13
- 7.8 KM . . . 13
- 7.9 最小平均環 . . . 13
- 7.10 floyd+ 負環偵測 . . . 14
- 7.11 Tarjan . . . 14
- 7.12 Topological Sort . . . 14
- 7.13 LCA . . . 14
- 7.14 樹鏈剖分 HLD . . . 15
- 7.15 虛樹 . . . 15
- 7.16 重心分解 . . . 15
- 7.17 2-SAT . . . 15
- 7.18 歐拉路徑 . . . 16
- 7.19 一般圖匹配 . . . 16
- 7.20 匹配相關定理 . . . 17
- 7.21 Dominator Tree . . . 17
- 7.22 Stoer-Wagner 全域最小割 . . . 17

8 Data Structure 17
- 8.1 2D Range Tree . . . 17
- 8.2 Segment Tree . . . 17
- 8.3 ZKW 線段樹 . . . 18
- 8.4 Sparse Table . . . 18
- 8.5 Lazy Tag . . . 18
- 8.6 Lazy Min / 線段樹上二分 . . . 18
- 8.7 BIT 樹狀樹組 . . . 19
- 8.8 並查集 union find . . . 19
- 8.9 可撤銷並查集 . . . 19
- 8.10 Treap . . . 19
- 8.11 2D BIT . . . 20
- 8.12 持久化線段樹 . . . 20
- 8.13 李超線段樹 . . . 21

9 String 21
- 9.1 KMP . . . 21
- 9.2 smallest rotation . . . 21
- 9.3 Suffix Array . . . 21
- 9.4 Z-value . . . 22
- 9.5 旋轉哈希 . . . 22
- 9.6 後綴自動機 . . . 22
- 9.7 AC 自動機 . . . 22
- 9.8 回文樹 . . . 23

10 Others 23
- 10.1 1D/1D dp 優化 . . . 23
- 10.2 Theorem - DP optimization . . . 23
- 10.3 Stable Marriage . . . 24
- 10.4 莫隊 . . . 24
- 10.5 莫隊帶修改 . . . 24
- 10.6 矩陣乘法 . . . 24
- 10.7 c++ 小抄 . . . 24
- 10.8 python 小抄 . . . 24
- 10.9 萬年曆 . . . 25
- 10.10 多重背包 . . . 25
- 10.11 LIS . . . 25

1 C++20

1.1 format / println

```cpp
#include <format>

template<class... t>
void println(format_string<t...> fmt, t&&... args) {
    cout << format(fmt, forward<t>(args)...) << '\n';
}

/*
format("{} + {} = {}", 2, 3, 5); // "2 + 3 = 5"
format("{1} {0}", "a", "b");     // "b a"

double x = 3.1415926;
format("{:.3f}", x);  // "3.142": fixed, 3 digits after
  decimal point
format("{:.3}", x);   // "3.14": 3 significant digits
format("{:8.2f}", x); // width 8, fixed 2 digits

format("{:<8}", "hi"); // left align
format("{:^8}", "hi"); // center
format("{:>8}", "hi"); // right align
format("{:08}", 42);   // "00000042"
format("{:+}", 42);    // "+42"
format("{:#x}", 255);  // "0xff"
format("{{{}}}", 42);  // "{42}"

println("answer = {}", 42);
println("pi = {:.6f}", numbers::pi);
*/
```

2 Basic

2.1 debug list

記得測試 python 的內建函數庫有哪些
bits/std++.h 跟 global variable y1 衝突，不能用

模板要記得 init
priority_queue 要清空
事先將把邊界測資加入測試
邊界條件 (過程溢位, 題目數據範圍), 會不會爆 long long
是否讀錯題目, 想不到時可以自己讀一次題目
比較容易有問題的地方換人寫
注意公式有沒有推錯或抄錯
精度誤差 sqrt(大大的東西) + EPS
喇分 random_suffle 隨機演算法

2.2 SVG 視覺化 debug

```cpp
#ifdef LOCAL // g++ -DLOCAL; open the .svg in a browser
struct svg { // shows [x1,x2] x [y1,y2], y axis points up
    ofstream o;
    string c = "red"; // current color
    double X, Y, k;    // map to [0, 1000] for the browser
    void p(string_view s) { o << s; }
    void p(string_view s, auto v, auto... vs) {
        auto i = s.find('$');
        o << s.substr(0, i) << v;
        p(s.substr(i + 1), vs...);
    }
    double tx(double x) { return (x - X) * k; }
    double ty(double y) { return (Y - y) * k; }
    svg(string f, double x1, double y1, double x2,
        double y2) : o(f), X(x1), Y(y2) {
        k = 1e3 / max(x2 - x1, y2 - y1);
        p("<svg xmlns='http://www.w3.org/2000/svg' "
          "viewBox='-50 -50 $ $'><style>"
          "*{stroke-width:.3%;fill:none}"
          "text{fill:black;text-anchor:middle;"
          "dominant-baseline:central}</style>\n",
          tx(x2) + 100, ty(y1) + 100);
    }
    ~svg() { p("</svg>\n"); }
    void line(double x1, double y1, double x2, double y2) {
        p("<line x1='$' y1='$' x2='$' y2='$' "
          "stroke='$'/>\n",
          tx(x1), ty(y1), tx(x2), ty(y2), c);
    }
    void circle(double x, double y, double r,
                string fill = "none") {
        p("<circle cx='$' cy='$' r='$' stroke='$' style="
          "'fill:$'/>\n", tx(x), ty(y), r * k, c, fill);
    }
    void text(double x, double y, auto s, double size) {
        p("<text x='$' y='$' font-size='$'>$</text>\n",
          tx(x), ty(y), size * k, s);
    }
    // svg(f, -1.2, -1.2, 1.2, 1.2); small ring = head
    void graph(int n, const vector<pair<int, int>> &e) {
        double t = 2 * acos(-1) / n;
        auto x = [&](double i) { return cos(t * i); };
        auto y = [&](double i) { return sin(t * i); };
        for (auto [u, v] : e) {
            line(x(u), y(u), x(v), y(v));
            circle(lerp(x(u), x(v), .85),
                   lerp(y(u), y(v), .85), .02);
        }
        for (int i = 0; i < n; ++i)
            circle(x(i), y(i), .07, "#fff"),
            text(x(i), y(i), i, .08);
    }
};
#else
struct svg {
    string c;
    svg(auto...) {}
    void line(auto...) {}
    void circle(auto...) {}
    void text(auto...) {}
    void graph(auto...) {}
};
#endif
```

2.3 時間複雜度

時間複雜度	可處理的最大 N 數量級（約）
$O(1)$	幾乎沒限制
$O(\log N)$	10^{18} 級別（如快速冪）
$O(\sqrt{N})$	10^{10}
$O(N)$	10^8
$O(N \log N)$	$10^6 \sim 5 \times 10^6$
$O(N\sqrt{N})$	$1 \times 10^5 \sim 2 \times 10^5$
$O(N^2)$	$3 \times 10^3 \sim 10^4$
$O(N^2 \log N)$	$10^3 \sim 3 \times 10^3$
$O(N^3)$	$300 \sim 500$
$O(2^N)$	$N \leq 20$
$O(N!)$	$N \leq 10$

3 Dark Code

3.1 IO optimization

```cpp
// 用 fread 一次讀一大塊的快速輸入，輸入量極大 (10^6 以
// 上) 時取代 cin。read(x) 遇到 EOF 回傳 false。
// fread input. read(x) returns false on eof.
struct fast_input {
    static constexpr size_t size = 1 << 16;
    array<char, size> buf{};
    size_t pos = 0, len = 0;

    int get() {
        if (pos == len) {
            len = fread(buf.data(), 1, size, stdin);
            pos = 0;
            if (!len) return EOF;
        }
        return buf[pos++];
    }

    template<class t>
    bool read(t &x) {
        int c = get(), sign = 1;
        while (c != EOF && c != '-' &&
               (c < '0' || c > '9'))
            c = get();
        if (c == EOF) return false;
        if (c == '-') sign = -1, c = get();
        x = 0;
        while ('0' <= c && c <= '9')
            x = x * 10 + c - '0', c = get();
        x *= sign;
        return true;
    }
};
```

4 Geometry

4.1 2D point

```cpp
// 二維點／向量基本運算：加減、數乘、內積 dot、外積
// cross、長度。其他幾何模板都依賴它。
using ld = long double;
const ld eps = 1e-12L;

struct point {
    ld x = 0, y = 0;
};

point operator+(point a, point b) {
    return {a.x + b.x, a.y + b.y};
}
point operator-(point a, point b) {
    return {a.x - b.x, a.y - b.y};
}
point operator*(point p, ld k) {
    return {p.x * k, p.y * k};
}
ld dot(point a, point b) { return a.x * b.x + a.y * b.y; }
ld cross(point a, point b) {
    return a.x * b.y - a.y * b.x;
```

```
}
ld norm2(point p) { return dot(p, p); }
ld norm(point p) { return sqrtl(norm2(p)); }
```

4.2 兩線段交點

```
// 整數座標判斷兩線段是否相交（含端點接觸、共線重疊），
// 用 __int128 避免溢位。
using i128 = __int128_t;

struct ipoint {
    long long x, y;
};

i128 cross(ipoint a, ipoint b, ipoint c) {
    return (i128(b.x) - a.x) * (i128(c.y) - a.y) -
           (i128(b.y) - a.y) * (i128(c.x) - a.x);
}

int sign(i128 x) { return (x > 0) - (x < 0); }

bool on_segment(ipoint a, ipoint b, ipoint p) {
    return cross(a, b, p) == 0 &&
           min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
           min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}

bool segments_intersect(ipoint a, ipoint b, ipoint c,
                        ipoint d) {
    i128 x = cross(a, b, c), y = cross(a, b, d);
    i128 z = cross(c, d, a), w = cross(c, d, b);
    if (!x && on_segment(a, b, c)) return true;
    if (!y && on_segment(a, b, d)) return true;
    if (!z && on_segment(c, d, a)) return true;
    if (!w && on_segment(c, d, b)) return true;
    return sign(x) != sign(y) && sign(z) != sign(w);
}
```

4.3 兩圓交點

```
// 求兩圓交點：回傳交點個數 (0/1/2，完全重合為 -1) 與交
// 點座標。
// count: -1 identical circles; otherwise number of points
struct circle_hit {
    int count;
    vector<point> p;
};

circle_hit circle_intersections(point a, ld r, point b,
                                ld s) {
    point d = b - a;
    ld d2 = norm2(d), len = sqrtl(d2);
    if (len <= eps && fabsl(r - s) <= eps) return {-1, {}};
    if (len <= eps || len > r + s + eps ||
        len < fabsl(r - s) - eps)
        return {0, {}};

    ld x = (d2 + r * r - s * s) / (2 * len);
    ld h2 = max<ld>(0, r * r - x * x);
    point mid = a + d * (x / len);
    if (h2 <= eps) return {1, {mid}};
    point off{-d.y * sqrtl(h2) / len,
              d.x * sqrtl(h2) / len};
    return {2, {mid + off, mid - off}};
}
```

4.4 Convex Hull

```
// Andrew 單調鏈求凸包，O(n log n)，回傳逆時針頂點、不含
// 共線點。
// needs point, cross and eps from 2Dpoint.cpp
vector<point> convex_hull(vector<point> p) {
    ranges::sort(p, [](point a, point b) {
        return tie(a.x, a.y) < tie(b.x, b.y);
    });
    auto same = [](point a, point b) {
        return norm2(a - b) <= eps * eps;
    };
    p.erase(ranges::unique(p, same).begin(), p.end());
    if (p.size() <= 1) return p;

    vector<point> h;
    for (int pass = 0; pass < 2; ++pass) {
        size_t base = h.size();
        for (point q : p) {
            while (h.size() >= base + 2 &&
                   cross(h.back() - h.end()[-2],
                         q - h.back()) <= eps)
                h.pop_back();
            h.push_back(q);
        }
        h.pop_back();
        ranges::reverse(p);
    }
    return h; // ccw; first point is not repeated
}
```

4.5 多邊形 / 旋轉卡尺

```
// 多邊形有向面積、點在多邊形內外判斷、點到線段距離；旋
// 轉卡尺求凸包直徑（最遠點對）。
// needs 2Dpoint.cpp; polygon vertices are given in order
// signed area, ccw is positive
ld polygon_area(const vector<point> &p) {
    ld s = 0;
    for (int i = 0, n = p.size(); i < n; ++i)
        s += cross(p[i], p[(i + 1) % n]);
    return s / 2;
}

ld segment_distance(point a, point b, point q) {
    point d = b - a;
    if (norm2(d) <= eps) return norm(q - a);
    ld t = clamp(dot(q - a, d) / norm2(d), ld(0), ld(1));
    return norm(q - (a + d * t));
}

// 1 inside, 0 on boundary, -1 outside
int point_in_polygon(const vector<point> &p, point q) {
    bool in = false;
    for (int i = 0, n = p.size(); i < n; ++i) {
        point a = p[i], b = p[(i + 1) % n];
        if (segment_distance(a, b, q) <= eps) return 0;
        if ((a.y > q.y) == (b.y > q.y)) continue;
        ld t = (q.y - a.y) / (b.y - a.y);
        if (q.x < a.x + (b.x - a.x) * t) in = !in;
    }
    return in ? 1 : -1;
}

// squared diameter by rotating calipers; h is ccw from
  convex_hull
ld diameter2(const vector<point> &h) {
    int n = h.size();
    ld best = 0;
    for (int i = 0, j = 1 % max(n, 1); i < n; ++i) {
        point e = h[(i + 1) % n] - h[i];
        while (cross(e, h[(j + 1) % n] - h[j]) > eps)
            j = (j + 1) % n;
        best = max({best, norm2(h[i] - h[j]),
                    norm2(h[(i + 1) % n] - h[j])});
    }
    return best;
}
```

4.6 最近點對

```
// 平面最近點對，掃描線 + set，O(n log n)，回傳距離平方。
// needs 2Dpoint.cpp; O(n log n), returns the squared
  distance, n >= 2
ld closest_pair(vector<point> p) {
    ranges::sort(p, {}, &point::x);
    set<pair<ld, ld>> s; // (y, x)
    ld best = numeric_limits<ld>::max();
    for (int i = 0, j = 0; i < ssize(p); ++i) {
        ld d = sqrtl(best);
        while (p[i].x - p[j].x > d)
```

```
            s.erase({p[j].y, p[j].x}), ++j;
        ld top = p[i].y + d;
        auto it = s.lower_bound({p[i].y - d, -HUGE_VALL});
        for (; it != s.end() && it->first <= top; ++it) {
            point q{it->second, it->first};
            best = min(best, norm2(p[i] - q));
        }
        s.insert({p[i].y, p[i].x});
    }
    return best;
}
```

4.7 外心

```
// 三角形外心（三點共圓的圓心），三點不可共線。
// needs 2Dpoint.cpp; a, b, c must not be collinear
point circumcentre(point a, point b, point c) {
    point u = b - a, v = c - a;
    ld d = 2 * cross(u, v);
    point o{v.y * norm2(u) - u.y * norm2(v),
            u.x * norm2(v) - v.x * norm2(u)};
    return a + o * (1 / d);
}
```

4.8 最小覆蓋圓

```
// 最小覆蓋圓：包住所有點的最小圓，隨機增量法期望 O(n)。
// needs circumcentre.cpp; expected O(n); returns {centre,
  radius}
pair<point, ld> smallest_circle(vector<point> p) {
    ranges::shuffle(p, mt19937(random_device{}()));
    point c;
    ld r2 = 0;
    auto outside = [&](point q) {
        return norm2(q - c) > r2 + eps;
    };
    for (int i = 0; i < ssize(p); ++i) if (outside(p[i])) {
        c = p[i], r2 = 0;
        for (int j = 0; j < i; ++j) if (outside(p[j])) {
            c = (p[i] + p[j]) * 0.5, r2 = norm2(p[i] - c);
            for (int k = 0; k < j; ++k) {
                if (!outside(p[k])) continue;
                c = circumcentre(p[i], p[j], p[k]);
                r2 = norm2(p[i] - c);
            }
        }
    }
    return {c, sqrtl(r2)};
}
```

4.9 半平面交

```
// 半平面交：求所有有向直線左側區域的交集（凸多邊形），
// 可解多邊形核、線性約束可行域。
// needs 2Dpoint.cpp; keeps the left side of every directed
  line
// returns a ccw polygon, empty if the area is empty
// the result may be unbounded: add a large bounding box
  first
struct line {
    point p, v; // p + t * v
    ld angle() const { return atan2l(v.y, v.x); }
};
bool on_left(line l, point q) {
    return cross(l.v, q - l.p) > eps;
}
point line_intersection(line a, line b) {
    return a.p +
           a.v * (cross(b.v, b.p - a.p) / cross(b.v, a.v));
}

vector<point> half_plane_intersection(vector<line> l) {
    ranges::sort(l, {}, &line::angle);
    int n = l.size(), first = 0, last = 0;
    vector<point> p(n);
    vector<line> q(n);
    q[0] = l[0];
    for (int i = 1; i < n; ++i) {
        while (first < last && !on_left(l[i], p[last - 1]))
            --last;
        while (first < last && !on_left(l[i], p[first]))
            ++first;
        q[++last] = l[i];
        // parallel: keep the inner one
        ld c = cross(q[last].v, q[last - 1].v);
        if (fabsl(c) <= eps) {
            --last;
            if (on_left(q[last], l[i].p)) q[last] = l[i];
        }
        if (first < last)
            p[last - 1] =
                line_intersection(q[last - 1], q[last]);
    }
    while (first < last && !on_left(q[first], p[last - 1]))
        --last;
    if (last - first <= 1) return {};
    p[last] = line_intersection(q[last], q[first]);
    return vector(p.begin() + first, p.begin() + last + 1);
}
```

4.10 凸包查詢 / 圓切線 / Minkowski

```
// 凸包 O(log n) 點包含查詢、直線與圓交點、點到圓的切點、
// Minkowski 和（兩凸包相加）。
// needs 2Dpoint.cpp
int sign(ld x) { return (x > eps) - (x < -eps); }

// h: ccw convex, no three collinear, size >= 3, O(log n)
// 1 inside, 0 on boundary, -1 outside
int in_convex(const vector<point> &h, point q) {
    int n = h.size(), l = 1, r = n - 1;
    auto side = [&](point a, point b) {
        return sign(cross(b - a, q - a));
    };
    if (side(h[0], h[1]) < 0 || side(h[0], h[n - 1]) > 0)
        return -1;
    while (r - l > 1) {
        int m = (l + r) / 2;
        (side(h[0], h[m]) >= 0 ? l : r) = m;
    }
    int s = side(h[l], h[l + 1]);
    if (s <= 0) return s;
    if (l == 1 && !side(h[0], h[1])) return 0;
    if (l == n - 2 && !side(h[0], h[n - 1])) return 0;
    return 1;
}

// line through a != b and circle (o, r)
vector<point> circle_line(point o, ld r, point a,
                          point b) {
    point d = b - a;
    point h = a + d * (dot(o - a, d) / norm2(d)); // foot
    ld s = r * r - norm2(o - h);
    if (s < -eps) return {};
    if (s <= eps) return {h};
    point off = d * (sqrtl(s) / norm(d));
    return {h - off, h + off};
}

// tangent points on circle (o, r) seen from p
vector<point> tangents(point o, ld r, point p) {
    point d = p - o;
    ld d2 = norm2(d);
    if (d2 < r * r - eps) return {};
    if (d2 <= r * r + eps) return {p};
    ld a = r * r / d2, b = r * sqrtl(d2 - r * r) / d2;
    point perp{-d.y, d.x};
    return {o + d * a + perp * b, o + d * a - perp * b};
}

// p, q: ccw convex polygons; result is ccw convex
vector<point> minkowski(vector<point> p, vector<point> q) {
    auto fix = [](vector<point> &v) {
        ranges::rotate(v, ranges::min_element(v, {},
            [](point a) { return pair(a.y, a.x); }));
        v.push_back(v[0]), v.push_back(v[1]);
    };
    fix(p), fix(q);
```

```
    vector<point> res;
    for (size_t i = 0, j = 0;
         i + 2 < p.size() || j + 2 < q.size();) {
        res.push_back(p[i] + q[j]);
        ld c = cross(p[i + 1] - p[i], q[j + 1] - q[j]);
        if (c >= -eps && i + 2 < p.size()) ++i;
        if (c <= eps && j + 2 < q.size()) ++j;
    }
    return res;
}
```

5 Flow

5.1 Dinic

```
// Dinic 最大流，另可取得最小割的 S 側點集。二分圖匹配、
// 最小割建模都用它。
using ll = long long;
const ll inf = LLONG_MAX / 4;

struct dinic {
    struct edge { int to, rev; ll cap; };
    vector<vector<edge>> g;
    vector<int> level, it;

    dinic(int n) : g(n), level(n), it(n) {}
    void add_edge(int u, int v, ll cap) {
        int a = g[u].size(), b = g[v].size();
        g[u].push_back({v, b, cap});
        g[v].push_back({u, a, 0});
    }
    bool bfs(int s, int t) {
        ranges::fill(level, -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto &e : g[u])
                if (e.cap && level[e.to] == -1)
                    level[e.to] = level[u] + 1,
                    q.push(e.to);
        }
        return level[t] != -1;
    }
    ll dfs(int u, int t, ll flow) {
        if (u == t) return flow;
        for (int &i = it[u]; i < ssize(g[u]); ++i) {
            edge &e = g[u][i];
            if (!e.cap || level[e.to] != level[u] + 1)
                continue;
            ll got = dfs(e.to, t, min(flow, e.cap));
            if (!got) continue;
            e.cap -= got;
            g[e.to][e.rev].cap += got;
            return got;
        }
        return 0;
    }
    ll max_flow(int s, int t) {
        ll ans = 0, got;
        while (bfs(s, t)) {
            ranges::fill(it, 0);
            while ((got = dfs(s, t, inf))) ans += got;
        }
        return ans;
    }
    vector<char> min_cut_side(int s) {
        vector<char> seen(g.size());
        vector<int> stack{ s };
        seen[s] = true;
        while (!stack.empty()) {
            int u = stack.back();
            stack.pop_back();
            for (auto &e : g[u]) if (e.cap && !seen[e.to])
                seen[e.to] = true, stack.push_back(e.to);
        }
        return seen;
    }
};

/*
lower bounds [low, high]:
add u->v with high-low; balance[u]-=low, balance[v]+=low.
add ss->v for positive balance, v->tt for negative balance.
for s-t flow also add t->s with inf before the feasibility
  flow.
after feasibility, remove ss/tt and t->s:
max = base + max_flow(s,t), min = base - max_flow(t,s).
*/
```

5.2 min cost flow

```
// 最小費用最大流（SPFA 找最短增廣路），可設流量上限；回
// 傳 {流量, 費用}。支援負費用邊。
using ll = long long;
const ll inf = LLONG_MAX / 4;

struct min_cost_flow {
    struct edge { int to, rev; ll cap, cost; };
    vector<vector<edge>> g;

    min_cost_flow(int n) : g(n) {}
    void add_edge(int u, int v, ll cap, ll cost) {
        int a = g[u].size(), b = g[v].size();
        g[u].push_back({v, b, cap, cost});
        g[v].push_back({u, a, 0, -cost});
    }
    pair<ll, ll> solve(int s, int t, ll limit = inf) {
        ll flow = 0, cost = 0;
        int n = g.size();
        vector<ll> dist(n);
        vector<int> pv(n), pe(n);
        vector<char> in_queue(n);
        while (flow < limit) {
            ranges::fill(dist, inf);
            ranges::fill(in_queue, false);
            queue<int> q;
            dist[s] = 0;
            q.push(s);
            in_queue[s] = true;
            while (!q.empty()) {
                int u = q.front();
                q.pop();
                in_queue[u] = false;
                for (int i = 0; i < ssize(g[u]); ++i) {
                    edge &e = g[u][i];
                    if (!e.cap) continue;
                    auto x = (__int128)dist[u] + e.cost;
                    ll nd = clamp<__int128>(x, -inf, inf);
                    if (dist[e.to] <= nd) continue;
                    dist[e.to] = nd;
                    pv[e.to] = u, pe[e.to] = i;
                    if (in_queue[e.to]) continue;
                    q.push(e.to), in_queue[e.to] = true;
                }
            }
            if (dist[t] == inf) break;
            ll add = limit - flow;
            for (int v = t; v != s; v = pv[v])
                add = min(add, g[pv[v]][pe[v]].cap);
            for (int v = t; v != s; v = pv[v]) {
                edge &e = g[pv[v]][pe[v]];
                e.cap -= add;
                g[v][e.rev].cap += add;
            }
            flow += add;
            cost += add * dist[t];
        }
        return {flow, cost};
    }
};
```

5.3 Gomory-Hu Tree

```
// Gomory-Hu 樹：無向圖做 n-1 次最大流建樹，任兩點最小割
// = 樹上路徑最小邊權。適合多組兩點最小割詢問。
```

```cpp
// needs dinic.cpp; undirected graph, n - 1 max flows
  (Gusfield)
// returns tree edges {u, v, w}; min cut(a, b) = min w on
  the tree path
vector<tuple<int, int, ll>> gomory_hu(
    int n, const vector<tuple<int, int, ll>> &edges) {
    vector<int> p(n);
    vector<tuple<int, int, ll>> tree;
    for (int s = 1; s < n; ++s) {
        dinic d(n);
        for (auto [u, v, w] : edges)
            d.add_edge(u, v, w), d.add_edge(v, u, w);
        ll f = d.max_flow(s, p[s]);
        auto side = d.min_cut_side(s);
        for (int i = s + 1; i < n; ++i)
            if (p[i] == p[s] && side[i]) p[i] = s;
        tree.push_back({s, p[s], f});
    }
    return tree;
}
```

6 Mathematics

6.1 ax+by=gcd(a,b)

```cpp
// 擴展歐幾里得：求 ax+by=gcd(a,b) 的一組整數解 (x,y)。
// 用途：求模反元素 (gcd(a,m)=1 時 x 即 a^(-1) mod m)、
// 解線性同餘 ax ≡ b (mod m)、exCRT 的基礎。
// 通解：x+k*b/g, y-k*a/g。
using ll = long long;

// returns {g, x, y}: ax + by = g = gcd(a,b)
tuple<ll, ll, ll> ext_gcd(ll a, ll b) {
    if (!b) return {abs(a), a < 0 ? -1 : 1, 0};
    ll g, x, y;
    tie(g, x, y) = ext_gcd(b, a % b);
    return {g, y, x - a / b * y};
}
```

6.2 exCRT 中國剩餘定理

```cpp
// 擴展中國剩餘定理：解聯立同餘 x ≡ r_i (mod m_i)，模數
// 「不必兩兩互質」。
// 逐條合併，回傳 {最小非負解 x, lcm}，無解回傳 nullopt；
// 通解為 x+k*lcm。
// 注意 lcm 必須在 long long 範圍內。
// needs ext_gcd from ax+by=gcd.cpp
// x = r[i] (mod m[i]), moduli need not be coprime; lcm must
  fit in ll
// returns {x, lcm} with 0 <= x < lcm, nullopt if no
  solution
optional<pair<ll, ll>> crt(const vector<ll> &r,
                           const vector<ll> &m) {
    ll x = 0, l = 1;
    for (int i = 0; i < ssize(r); ++i) {
        auto [g, p, q] = ext_gcd(l, m[i]);
        ll d = (r[i] % m[i] + m[i]) % m[i] - x;
        if (d % g) return nullopt;
        ll mg = m[i] / g;
        ll k = (__int128)(d / g % mg) * p % mg;
        ll nl = l / g * m[i];
        x = ((x + (__int128)l * k) % nl + nl) % nl;
        l = nl;
    }
    return pair{x, l};
}
```

6.3 BSGS 離散對數

```cpp
// 擴展 BSGS 離散對數：求最小 x >= 0 使 a^x ≡ b (mod m)，
// m 不必是質數，O(sqrt(m))。
// 先不斷除掉 gcd(a,m) 化成互質情形，再用大步小步 (hash
// 表) 碰撞。無解回傳 -1。
using ll = long long;

// smallest x >= 0 with a^x = b (mod m); m need not be
  prime; -1 if none
// baby-step giant-step, O(sqrt(m))
ll discrete_log(ll a, ll b, ll m) {
    a %= m, b %= m;
    ll k = 1 % m, add = 0;
    for (ll g; (g = gcd(a, m)) > 1;) {
        if (b == k) return add;
        if (b % g) return -1;
        b /= g, m /= g, ++add;
        k = (__int128)k * (a / g) % m;
    }
    if (b == k) return add;
    // now k * a^x = b (mod m) with gcd(a, m) = 1
    ll n = sqrtl(m) + 1, an = 1;
    unordered_map<ll, ll> seen;
    for (ll q = 0, cur = b; q < n; ++q)
        seen[cur] = q, cur = (__int128)cur * a % m;
    for (int i = 0; i < n; ++i) an = (__int128)an * a % m;
    for (ll p = 1, cur = k; p <= n; ++p) {
        cur = (__int128)cur * an % m;
        if (auto it = seen.find(cur); it != seen.end())
            return n * p - it->second + add;
    }
    return -1;
}
```

6.4 原根

```cpp
// 求奇質數 p 的最小原根 g（g 的冪可走遍 1..p-1）。
// 做法：分解 p-1，對每個質因數 q 檢查 g^((p-1)/q) != 1。
// 用途：NTT 找 root、把乘法轉成指數加法（離散對數／ k 次
// 剩餘）。
// needs mod_power from number_tool.cpp; p is an odd prime
ll primitive_root(ll p) {
    vector<ll> f;
    ll n = p - 1;
    for (ll i = 2; i * i <= n; ++i) if (n % i == 0) {
        f.push_back(i);
        while (n % i == 0) n /= i;
    }
    if (n > 1) f.push_back(n);
    for (ll g = 2;; ++g)
        if (ranges::all_of(f, [&](ll q) {
                return mod_power(g, (p - 1) / q, p) != 1;
            })) return g;
}
```

6.5 GaussElimination

```cpp
// 高斯消去法解實數線性方程組（輸入增廣矩陣），回傳 0 無
// 解 / 1 唯一解 / 2 無限多解。
using ld = long double;
const ld eps = 1e-12L;

// augmented matrix; returns 0=no solution, 1=unique,
  2=infinite
int gauss(vector<vector<ld>> a, vector<ld> &answer) {
    int n = a.size(), m = a.empty() ? 0 : a[0].size() - 1,
        row = 0;
    vector<int> where(m, -1);
    for (int col = 0; col < m && row < n; ++col) {
        int best = row;
        for (int i = row; i < n; ++i)
            if (fabsl(a[i][col]) > fabsl(a[best][col]))
                best = i;
        if (fabsl(a[best][col]) <= eps) continue;
        swap(a[best], a[row]);
        where[col] = row;
        ld div = a[row][col];
        for (int j = col; j <= m; ++j) a[row][j] /= div;
        for (int i = 0; i < n; ++i) if (i != row) {
            ld mul = a[i][col];
            for (int j = col; j <= m; ++j)
                a[i][j] -= mul * a[row][j];
        }
        ++row;
    }
    answer.assign(m, 0);
    for (int i = 0; i < m; ++i) if (where[i] != -1)
        answer[i] = a[where[i]][m];
```

```
    for (auto &r : a) {
        ld sum = 0;
        for (int i = 0; i < m; ++i)
            sum += r[i] * answer[i];
        if (fabsl(sum - r[m]) > eps) return 0;
    }
    return ranges::count(where, -1) ? 2 : 1;
}
```

6.6 Inverse

```
// 模反元素：快速冪 a^(p-2)（費馬小定理，p 為質數）；
// 以及 O(n) 線性求 1..n 所有反元素：inv[i] =
// -floor(p/i)*inv[p mod i]。
// 模數非質數時改用擴展歐幾里得。
using ll = long long;

ll power(ll a, ll n, ll mod) {
    ll ans = 1;
    for (; n; n /= 2, a = (__int128)a * a % mod)
        if (n & 1) ans = (__int128)ans * a % mod;
    return ans;
}

// mod must be prime; 1 <= n < mod
vector<ll> inverse_table(int n, ll mod) {
    assert(1 <= n && n < mod);
    vector<ll> inv(n + 1);
    inv[1] = 1;
    for (int i = 2; i <= n; ++i)
        inv[i] =
            mod - (__int128)(mod / i) * inv[mod % i] % mod;
    return inv;
}
```

6.7 LinearPrime 歐拉篩

```
// 線性篩（歐拉篩）O(n) 求 <= n 的所有質數，每個合數只被
// 最小質因數篩一次。
// 內部 min_prime[] 是最小質因數，留著可 O(log n) 質因數
// 分解；同架構也能順便篩 phi、mu 等積性函數。
vector<int> linear_sieve(int n) {
    vector<int> primes, min_prime(n + 1);
    for (int i = 2; i <= n; ++i) {
        if (!min_prime[i])
            min_prime[i] = i, primes.push_back(i);
        for (int p : primes) {
            if (p > min_prime[i] || (long long)i * p > n)
                break;
            min_prime[i * p] = p;
        }
    }
    return primes;
}
```

6.8 Miller Rabin

```
// Miller-Rabin 質數測試，使用 7 個固定底數，對所有 64
// 位元整數都是確定性正確。
// 附 __int128 的模乘 mod_mul 與快速冪 mod_power，供
// Pollard rho 使用。
using u64 = uint64_t;
using u128 = __uint128_t;

u64 mod_mul(u64 a, u64 b, u64 mod) {
    return u128(a) * b % mod;
}
u64 mod_power(u64 a, u64 n, u64 mod) {
    u64 ans = 1;
    for (; n; n /= 2, a = mod_mul(a, a, mod))
        if (n & 1) ans = mod_mul(ans, a, mod);
    return ans;
}

bool is_prime(u64 n) {
    if (n < 2 || n % 2 == 0) return n == 2;
    u64 d = n - 1, s = 0;
    while (!(d & 1)) d /= 2, ++s;
    for (u64 a : array<u64, 7>{2, 325, 9375, 28178, 450775,
                               9780504, 1795265022}) {
        if (a % n == 0) continue;
        u64 x = mod_power(a, d, n);
        if (x == 1 || x == n - 1) continue;
        bool composite = true;
        for (u64 r = 1; r < s; ++r) {
            x = mod_mul(x, x, n);
            if (x == n - 1) {
                composite = false;
                break;
            }
        }
        if (composite) return false;
    }
    return true;
}
```

6.9 Pollard’s rho

```
// Pollard's rho 大數質因數分解（期望約 O(n^(1/4))），可
// 分解到 10^18。
// factor(n, cnt) 把質因數與次數存進 map；需搭配
// Miller-Rabin 判斷質數。
// needs is_prime() and mod_mul() from Miller-Rabin.cpp
using u64 = uint64_t;

u64 pollard(u64 n) {
    if (!(n & 1)) return 2;
    static mt19937_64 rng(time(0));
    while (true) {
        u64 c = rng() % (n - 1) + 1;
        u64 x = rng() % n, y = x, d = 1;
        auto f = [&](u64 v) {
            return (__uint128_t(mod_mul(v, v, n)) + c) % n;
        };
        while (d == 1) {
            x = f(x), y = f(f(y));
            d = gcd(x > y ? x - y : y - x, n);
        }
        if (d != n) return d;
    }
}

void factor(u64 n, map<u64, int> &count) {
    if (n == 1) return;
    if (is_prime(n)) return ++count[n], void();
    u64 d = pollard(n);
    factor(d, count);
    factor(n / d, count);
}
```

6.10 NTT

```
// 數論轉換 (NTT)：模 998244353 下的多項式乘法／卷積，
// O(n log n)，結果精確無誤差。
// 係數需在 [0,mod)；換其他 NTT 模數要改對應原根 root。
// coefficients must be in [0, mod)
const int mod = 998244353, root = 3;

int power(int a, int n) {
    long long ans = 1;
    for (; n; n /= 2, a = (long long)a * a % mod)
        if (n & 1) ans = ans * a % mod;
    return ans;
}

void ntt(vector<int> &a, bool inverse) {
    int n = a.size();
    assert(n && !(n & (n - 1)) && (mod - 1) % n == 0);
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n / 2;
        for (; j & bit; bit /= 2) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len *= 2) {
        int wlen = power(root, (mod - 1) / len);
```

```
        if (inverse) wlen = power(wlen, mod - 2);
        for (int i = 0; i < n; i += len) {
            long long w = 1;
            for (int j = 0; j < len / 2; ++j) {
                int u = a[i + j],
                    v = w * a[i + j + len / 2] % mod;
                a[i + j] =
                    u + v < mod ? u + v : u + v - mod;
                a[i + j + len / 2] =
                    u - v >= 0 ? u - v : u - v + mod;
                w = w * wlen % mod;
            }
        }
    }
    if (inverse) {
        int inv = power(n, mod - 2);
        for (int &x : a) x = (long long)x * inv % mod;
    }
}

vector<int> convolution(vector<int> a, vector<int> b) {
    if (a.empty() || b.empty()) return {};
    int need = a.size() + b.size() - 1, n = 1;
    while (n < need) n *= 2;
    assert(n <= (1 << 23));
    a.resize(n), b.resize(n);
    ntt(a, false), ntt(b, false);
    for (int i = 0; i < n; ++i)
        a[i] = (long long)a[i] * b[i] % mod;
    ntt(a, true);
    a.resize(need);
    return a;
}
```

6.11 FFT

```
// 浮點 FFT 多項式乘法（整數係數），O(n log n)；結果每項
// 需小於約 10^15 才不會有精度誤差。
// floating fft; exact while every result value < about 1e15
using ll = long long;
using cd = complex<double>;

void fft(vector<cd> &a, bool inverse) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n / 2;
        for (; j & bit; bit /= 2) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    vector<cd> root(n / 2);
    for (int i = 0; i < n / 2; ++i)
        root[i] = polar(1.0, 2 * numbers::pi * i / n);
    for (int len = 2; len <= n; len *= 2)
        for (int i = 0; i < n; i += len)
            for (int j = 0; j < len / 2; ++j) {
                cd w = root[n / len * j];
                if (inverse) w = conj(w);
                cd &x = a[i + j], &y = a[i + j + len / 2];
                cd u = x, v = y * w;
                x = u + v, y = u - v;
            }
    if (inverse) for (cd &x : a) x /= n;
}

vector<ll> multiply(const vector<ll> &a,
                    const vector<ll> &b) {
    if (a.empty() || b.empty()) return {};
    int need = a.size() + b.size() - 1, n = 1;
    while (n < need) n *= 2;
    vector<cd> fa(a.begin(), a.end()),
        fb(b.begin(), b.end());
    fa.resize(n), fb.resize(n);
    fft(fa, false), fft(fb, false);
    for (int i = 0; i < n; ++i) fa[i] *= fb[i];
    fft(fa, true);
    vector<ll> c(need);
    for (int i = 0; i < need; ++i)
        c[i] = llround(fa[i].real());
    return c;
}
```

6.12 FWHT

```
// 快速沃爾什轉換：位元運算卷積 c[i op j] += a[i]b[j]，
// op 可為 xor / and / or，O(n log n)。
// or 正轉換即子集和 (SOS dp)，and 為超集和；在模數下要
// 把除以 n 改成乘反元素。
// c[i op j] += a[i] * b[j]; op is '^', '&' or '|'; size is
  a power of two
// under a modulus replace "/= n" with "*= inverse(n)"
// SOS dp = fwht(a, '|', false); superset sum uses '&'
using ll = long long;

void fwht(vector<ll> &a, char op, bool inverse) {
    int n = a.size();
    for (int len = 1; len < n; len *= 2)
        for (int i = 0; i < n; i += 2 * len)
            for (int j = i; j < i + len; ++j) {
                ll &x = a[j], &y = a[j + len];
                if (op == '^')
                    tie(x, y) = pair(x + y, x - y);
                else if (op == '&')
                    x += inverse ? -y : y; // superset sum
                else
                    y += inverse ? -x : x; // subset sum
            }
    if (inverse && op == '^') for (ll &x : a) x /= n;
}

vector<ll> fwht_convolution(vector<ll> a, vector<ll> b,
                            char op) {
    fwht(a, op, false), fwht(b, op, false);
    for (int i = 0; i < ssize(a); ++i) a[i] *= b[i];
    fwht(a, op, true);
    return a;
}
```

6.13 Simplex

```
// 單純形法解線性規劃：最大化 c^Tx，限制 Ax <= b, x >= 0；
// 回傳最優值，無界為 inf、無解為 -inf。
// maximize c^T x subject to A x <= b, x >= 0 (two-phase
  simplex)
// solve() returns the optimum, inf if unbounded, -inf if
  infeasible
using ld = long double;

struct simplex {
    static constexpr ld eps = 1e-9, inf = HUGE_VALL;
    int m, n;
    vector<int> N, B;
    vector<vector<ld>> D;

    simplex(const vector<vector<ld>> &A,
            const vector<ld> &b, const vector<ld> &c)
        : m(b.size()), n(c.size()), N(n + 1), B(m),
          D(m + 2, vector<ld>(n + 2)) {
        for (int i = 0; i < m; ++i) {
            for (int j = 0; j < n; ++j) D[i][j] = A[i][j];
            B[i] = n + i, D[i][n] = -1, D[i][n + 1] = b[i];
        }
        for (int j = 0; j < n; ++j)
            N[j] = j, D[m][j] = -c[j];
        N[n] = -1, D[m + 1][n] = 1;
    }
    void pivot(int r, int s) {
        ld inv = 1 / D[r][s];
        for (int i = 0; i < m + 2; ++i)
            if (i != r && fabsl(D[i][s]) > eps) {
                ld k = D[i][s] * inv;
                for (int j = 0; j < n + 2; ++j)
                    D[i][j] -= D[r][j] * k;
                D[i][s] = D[r][s] * k;
            }
        for (int j = 0; j < n + 2; ++j)
            if (j != s) D[r][j] *= inv;
        for (int i = 0; i < m + 2; ++i)
            if (i != r) D[i][s] *= -inv;
```

```
        D[r][s] = inv;
        swap(B[r], N[s]);
    }
    bool lt(int i, int a, int b) { // column a before b
        return pair(D[i][a], N[a]) < pair(D[i][b], N[b]);
    }
    bool run(int phase) {
        for (int x = m + phase - 1;;) {
            int s = -1;
            for (int j = 0; j <= n; ++j) {
                if (N[j] == -phase) continue;
                if (s == -1 || lt(x, j, s)) s = j;
            }
            if (D[x][s] >= -eps) return true;
            auto ratio = [&](int i) {
                return pair(D[i][n + 1] / D[i][s], B[i]);
            };
            int r = -1;
            for (int i = 0; i < m; ++i) if (D[i][s] > eps)
                if (r == -1 || ratio(i) < ratio(r)) r = i;
            if (r == -1) return false;
            pivot(r, s);
        }
    }
    ld solve(vector<ld> &x) {
        int r = 0;
        for (int i = 1; i < m; ++i)
            if (D[i][n + 1] < D[r][n + 1]) r = i;
        if (D[r][n + 1] < -eps) {
            pivot(r, n);
            if (!run(2) || D[m + 1][n + 1] < -eps)
                return -inf;
            for (int i = 0; i < m; ++i) if (B[i] == -1) {
                int s = 0;
                for (int j = 1; j <= n; ++j)
                    if (lt(i, j, s)) s = j;
                pivot(i, s);
            }
        }
        bool ok = run(1);
        x.assign(n, 0);
        for (int i = 0; i < m; ++i)
            if (B[i] < n) x[B[i]] = D[i][n + 1];
        return ok ? D[m][n + 1] : inf;
    }
};
```

6.14 線性基 XOR basis

```
// XOR 線性基：插入數字、判斷某數能否由集合 xor 出來、求
// 子集 xor 最大值。
// xor basis; b[i] has highest bit i
using ull = unsigned long long;

struct xor_basis {
    array<ull, 64> b{};
    bool insert(ull x) { // false if x is already spanned
        for (int i = 63; i >= 0; --i) if (x >> i & 1) {
            if (!b[i]) return b[i] = x, true;
            x ^= b[i];
        }
        return false;
    }
    bool contains(ull x) {
        for (int i = 63; i >= 0; --i) if (x >> i & 1) {
            if (!b[i]) return false;
            x ^= b[i];
        }
        return true;
    }
    ull max_xor(ull x = 0) {
        for (int i = 63; i >= 0; --i) x = max(x, x ^ b[i]);
        return x;
    }
};
```

6.15 Berlekamp-Massey / 線性遞迴第 k 項

```
// Berlekamp-Massey：給數列前 2L 項，求出最短線性遞迴式
// （模質數）；
// 再用 linear_rec 以多項式取模在 O(L^2 log k) 求第 k 項。
// 適合「打表發現是線性遞迴」的題目。
// mod must be prime; give about 2L terms for order L
using ll = long long;
const ll mod = 998244353;

ll power(ll a, ll n) {
    ll ans = 1;
    for (a %= mod; n; n /= 2, a = a * a % mod)
        if (n & 1) ans = ans * a % mod;
    return ans;
}

// returns c: s[i] = sum c[j] * s[i - 1 - j]
vector<ll> berlekamp_massey(const vector<ll> &s) {
    int n = s.size(), L = 0, m = 0;
    vector<ll> C(n + 1), B(n + 1), T;
    C[0] = B[0] = 1;
    ll b = 1;
    for (int i = 0; i < n; ++i) {
        ++m;
        ll d = s[i] % mod;
        for (int j = 1; j <= L; ++j)
            d = (d + C[j] * s[i - j]) % mod;
        if (!d) continue;
        T = C;
        ll coef = d * power(b, mod - 2) % mod;
        for (int j = m; j <= n; ++j)
            C[j] = (C[j] + (mod - coef) * B[j - m]) % mod;
        if (2 * L > i) continue;
        L = i + 1 - L, B = T, b = d, m = 0;
    }
    vector<ll> c(L);
    for (int j = 0; j < L; ++j)
        c[j] = (mod - C[j + 1]) % mod;
    return c;
}

// k-th term (0-based), s: first L terms, O(L^2 log k)
ll linear_rec(vector<ll> s, vector<ll> c, ll k) {
    int n = c.size();
    auto mul = [&](vector<ll> a, vector<ll> b) {
        vector<ll> r(2 * n + 1);
        for (int i = 0; i <= n; ++i)
            for (int j = 0; j <= n; ++j)
                r[i + j] = (r[i + j] + a[i] * b[j]) % mod;
        for (int i = 2 * n; i > n; --i)
            for (int j = 0; j < n; ++j) {
                ll &t = r[i - 1 - j];
                t = (t + r[i] * c[j]) % mod;
            }
        r.resize(n + 1);
        return r;
    };
    vector<ll> pol(n + 1), e(n + 1);
    pol[0] = e[1] = 1;
    for (++k; k; k /= 2) {
        if (k & 1) pol = mul(pol, e);
        e = mul(e, e);
    }
    ll ans = 0;
    for (int i = 0; i < n; ++i)
        ans = (ans + pol[i + 1] * s[i]) % mod;
    return ans;
}
```

6.16 Lagrange 插值

```
// 拉格朗日插值：已知 f(0),...,f(n-1) (deg f<n)，O(n) 求
// 任意 f(x) mod p。
// 常用於求 sum_{i=1..x} i^k 這類多項式前綴和（它是 k+1
// 次多項式）。
// needs power, mod from BerlekampMassey.cpp
// y[i] = f(i) in [0, mod) for i in [0, n), deg f < n
ll lagrange(const vector<ll> &y, ll x) {
    int n = y.size();
    x %= mod;
    vector<ll> pre(n + 1, 1), suf(n + 1, 1), fact(n, 1);
```

```
    for (int i = 0; i < n; ++i)
        pre[i + 1] = pre[i] * ((x - i + mod) % mod) % mod;
    for (int i = n - 1; i >= 0; --i)
        suf[i] = suf[i + 1] * ((x - i + mod) % mod) % mod;
    for (int i = 1; i < n; ++i)
        fact[i] = fact[i - 1] * i % mod;
    ll ans = 0;
    for (int i = 0; i < n; ++i) {
        ll t = y[i] * pre[i] % mod * suf[i + 1] % mod;
        ll d = fact[i] * fact[n - 1 - i] % mod;
        t = t * power(d, mod - 2) % mod;
        if ((n - 1 - i) % 2) t = (mod - t) % mod;
        ans = (ans + t) % mod;
    }
    return ans;
}
```

6.17 數論基本工具

```
// 數論常用小工具：快速冪 mod_power；
// 預處理階乘與階乘反元素，O(1) 求組合數 C(n,k) mod p（p
// 為質數且 n<p；n >= p 請用 Lucas）。
using ll = long long;

ll mod_power(ll a, ll n, ll mod) {
    ll ans = 1;
    for (; n; n /= 2, a = (__int128)a * a % mod)
        if (n & 1) ans = (__int128)ans * a % mod;
    return ans;
}

// mod must be prime and n < mod
struct combinations {
    ll mod;
    vector<ll> fact, inv_fact;
    combinations(int n, ll mod)
        : mod(mod), fact(n + 1, 1), inv_fact(n + 1) {
        for (int i = 1; i <= n; ++i)
            fact[i] = fact[i - 1] * i % mod;
        inv_fact[n] = mod_power(fact[n], mod - 2, mod);
        for (int i = n; i; --i)
            inv_fact[i - 1] = inv_fact[i] * i % mod;
    }
    ll choose(int n, int k) {
        if (k < 0 || k > n) return 0;
        return fact[n] * inv_fact[k] % mod *
               inv_fact[n - k] % mod;
    }
};
```

6.18 Mobius

```
// 線性篩求莫比烏斯函數 mu(1..n)。
// mu(n)：n=1 為 1；n 有平方因數為 0；否則為 (-1)^(質因
// 數個數)。
// 用於莫比烏斯反演，例如計算 gcd(i,j)=1 的對數：sum_d
// mu(d) floor(n/d) floor(m/d)，常搭配數論分塊。
vector<int> mobius(int n) {
    vector<int> mu(n + 1), primes;
    vector<char> composite(n + 1);
    if (n < 1) return mu;
    mu[1] = 1;
    for (int i = 2; i <= n; ++i) {
        if (!composite[i]) primes.push_back(i), mu[i] = -1;
        for (int p : primes) {
            if ((long long)i * p > n) break;
            composite[i * p] = true;
            if (i % p == 0) {
                mu[i * p] = 0;
                break;
            }
            mu[i * p] = -mu[i];
        }
    }
    return mu;
}
```

6.19 數論分塊 (Quotient Blocking)

```
// 數論分塊（整除分塊）：floor(n/i) 只有 O(sqrt(n)) 種值，
// 值相同的 i 連成一段，一段一起算。
// 一段從 l 開始：值 q = n / l，右端點 r = n / q。
// pre(x) = f(1) + ... + f(x)（f 的前綴和），pre(0) = 0；
// 每段貢獻 (pre(r) - pre(l - 1)) * q。
// 例：f = 1 時 pre = [](ll x) { return x; }
//     gcd(i, j) = 1 的對數：sum_floor_product 配 mu 前綴和
// 取模時把 pre 差值與乘法改成模運算。
using ll = long long;
using i128 = __int128;

// sum_{i=1..n} f(i) * floor(n / i)，n >= 0
i128 sum_floor(ll n, auto &&pre) {
    i128 ans = 0;
    for (ll l = 1, r; l <= n; l = r + 1) {
        ll q = n / l;
        r = n / q;
        ans += (i128)(pre(r) - pre(l - 1)) * q;
        if (r == n)
            break; // 避免 n == LLONG_MAX 時 r + 1 溢位
    }
    return ans;
}

// sum_{i=1..min(n,m)} f(i) * floor(n / i) * floor(m / i)
// 兩個值都要不變，右端點取 min
i128 sum_floor_product(ll n, ll m, auto &&pre) {
    ll limit = min(n, m);
    i128 ans = 0;
    for (ll l = 1, r; l <= limit; l = r + 1) {
        ll qn = n / l, qm = m / l;
        r = min(n / qn, m / qm);
        ans += (i128)(pre(r) - pre(l - 1)) * qn * qm;
        if (r == limit) break;
    }
    return ans;
}
```

6.20 SG

```
// SG 函數：在遊戲 DAG 上求每個狀態的 Grundy 值 (mex)；
// 多個獨立遊戲把 SG 值 xor，非 0 先手必勝。
/*
finite impartial game:
sg[u] = mex({sg[v] | u -> v}).
xor the sg values only when combining independent games.
normal play: xor != 0 wins.
misere nim: if every pile is 1, odd pile count loses;
  otherwise xor != 0 wins.
*/
vector<int> grundy(const vector<vector<int>> &g) {
    vector<int> sg(g.size(), -1);
    function<int(int)> dfs = [&](int u) {
        if (sg[u] != -1) return sg[u];
        vector<int> seen(g[u].size() + 1);
        for (int v : g[u]) {
            int x = dfs(v);
            if (x < ssize(seen)) seen[x] = true;
        }
        return sg[u] =
                    ranges::find(seen, 0) - seen.begin();
    };
    for (int u = 0; u < ssize(g); ++u) dfs(u);
    return sg;
}
```

6.21 歐拉定理

擴展歐拉定理（歐拉降冪）：指數 k 很大（如 a^{b^c}、冪塔）時，把指數對 $\varphi(m)$ 取模。
a, m 不互質時要注意 $k \geq \varphi(m)$ 需再加回 $\varphi(m)$。
歐拉降冪對於任何正整數 m、整數 a 和非負整數 k

$$a^k \equiv \begin{cases} a^{k \bmod \varphi(m)}, & \gcd(a,m) = 1, \\ a^k, & \gcd(a,m) \neq 1,\ k < \varphi(m), \\ a^{(k \bmod \varphi(m)) + \varphi(m)}, & \gcd(a,m) \neq 1,\ k \geq \varphi(m). \end{cases} \pmod m$$

6.22 Theorem

Lucas 定理（p 為質數）：把 n, m 寫成 p 進位，逐位算組合數再相乘。

$$\binom{n}{m} \equiv \binom{\lfloor n/p \rfloor}{\lfloor m/p \rfloor}\binom{n \bmod p}{m \bmod p} \pmod p$$

用途：n 很大（$\geq p$）時求 $\binom{n}{m} \bmod p$；遞迴到 $n < p$ 再用階乘表。

Pick 定理：頂點都在格子點上的簡單多邊形，面積 $A = i + \frac{b}{2} - 1$，i 為內部格子點數，b 為邊上格子點數。線段 (x_1, y_1)-(x_2, y_2) 上的格子點（不含一端）有 $\gcd(|\Delta x|, |\Delta y|)$ 個。

矩陣樹定理 (Kirchhoff)：求生成樹個數。無向圖：拉普拉斯矩陣 $L = D - A$（D 為度數對角矩陣，A 為鄰接矩陣，重邊要算次數），刪掉任意一行和同編號的一列，剩下的行列式就是生成樹個數。有向圖、以 r 為根的外向樹：用入度 $L = D_{in} - A$，刪掉第 r 行第 r 列。行列式在模質數下用高斯消去 $O(n^3)$。

Catalan 數：$C_0 = 1$，$C_{n+1} = C_n \cdot \frac{2(2n+1)}{n+2}$。（通項與數列見下一節組合數學公式）

莫比烏斯函數 μ：$\mu(1) = 1$；n 有平方因數時 $\mu(n) = 0$；$n = p_1 p_2 \cdots p_k$（相異質數）時 $\mu(n) = (-1)^k$。

- 積性：$\gcd(a,b) = 1 \Rightarrow \mu(ab) = \mu(a)\mu(b)$
- $\sum_{d|n} \mu(d) = [n = 1]$，所以 $[\gcd(i,j) = 1] = \sum_{d|\gcd(i,j)} \mu(d)$
- $\sum_{d|n} \varphi(d) = n$，所以 $\gcd(i,j) = \sum_{d|\gcd(i,j)} \varphi(d)$

莫比烏斯反演：

$$f(n) = \sum_{d|n} g(d) \iff g(n) = \sum_{d|n} \mu(d)\, f\left(\frac{n}{d}\right)$$

用途：算「$\gcd(i,j) = k$ 的對數」或 $\sum \gcd(i,j)$ 這類題目，反演後變成 $\sum_d \mu(d)\lfloor n/d \rfloor \lfloor m/d \rfloor$，再用數論分塊 $O(\sqrt{n})$。

中國剩餘定理（m_i 兩兩互質；不互質請用 exCRT 模板）：解 $x \equiv a_i \pmod{m_i}$。令 $M = \prod m_i$，$M_i = M/m_i$，$t_i = M_i^{-1} \bmod m_i$，則

$$x \equiv \sum_i a_i\, t_i\, M_i \pmod M$$

在模 M 下解唯一。

Burnside 引理：在對稱操作（旋轉、翻轉）下視為相同的方案數

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$$

即「每種操作下不動的方案數」的平均。X^g：做完操作 g 後不變的方案。例：n 顆珠子 k 種顏色的項鍊（只算旋轉）有 $\frac{1}{n}\sum_{d|n} \varphi(d)\, k^{n/d}$ 種。

Fermat / Euler / Wilson：p 質數時 $a^{p-1} \equiv 1 \pmod p$（$p \nmid a$）；$\gcd(a,m) = 1$ 時 $a^{\varphi(m)} \equiv 1 \pmod m$；$(p-1)! \equiv -1 \pmod p$。

6.23 組合數學公式

第一類 Stirling 數 $s(n,k)$：n 個相異元素排成 k 個環。$s(n,k) = s(n-1,k-1) + (n-1)\,s(n-1,k)$

第二類 Stirling 數 $S(n,k)$：n 個相異球放進 k 個相同的非空盒子。

$$S(n,k) = S(n-1,k-1) + k\,S(n-1,k) = \frac{1}{k!}\sum_{i=0}^{k}(-1)^i\binom{k}{i}(k-i)^n$$

下降冪展開：$x^n = \sum_k S(n,k)\, x(x-1)\cdots(x-k+1)$

Bell 數 $B_n = \sum_k S(n,k)$（集合分割數），$B_{n+1} = \sum_k \binom{n}{k} B_k$：1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975

Catalan 數 $C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$：1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796
合法括號序列、$n+2$ 邊形三角剖分、n 個節點的二元樹、不越過對角線的格子路徑數。

錯排 D_n（沒有人拿到自己的東西）：$D_n = (n-1)(D_{n-1} + D_{n-2})$，$D_0 = 1, D_1 = 0$；$D_n = n!\sum_{i=0}^{n} \frac{(-1)^i}{i!}$

分割數 $p(n)$（n 拆成正整數和、不計順序）：1, 1, 2, 3, 5, 7, 11, 15, 22, 30, 42；$p(100) = 190569292$

球與盒（n 個球、k 個盒子）：

	可以空盒	盒子非空
相同球、相異盒	$\binom{n+k-1}{k-1}$	$\binom{n-1}{k-1}$
相異球、相異盒	k^n	$k!\,S(n,k)$
相異球、相同盒	$\sum_{i=1}^{k} S(n,i)$	$S(n,k)$
相同球、相同盒	n 拆成至多 k 份	n 拆成恰 k 份

Cayley 公式：n 個有標號點的樹有 n^{n-2} 棵；完全二分圖 $K_{m,n}$ 的生成樹有 $m^{n-1}n^{m-1}$ 棵；n 點、k 棵樹且根為指定 k 點的森林有 $k\,n^{n-k-1}$ 個。

Prüfer 序列：長度 $n-2$，點 i 出現 $\deg(i) - 1$ 次；給定每點度數 d_i 的樹有 $\frac{(n-2)!}{\prod(d_i-1)!}$ 棵。

Hall 定理：二分圖左側能全部配對 $\iff$ 左側任意子集 S 滿足 $|N(S)| \geq |S|$。
Dilworth：偏序集的最少鏈覆蓋 = 最大反鏈大小；**Mirsky**：最少反鏈覆蓋 = 最長鏈長度。
Erdős–Gallai：$d_1 \geq \cdots \geq d_n$ 能成為簡單圖的度數序列 $\iff$ 總和為偶數，且對每個 k：$\sum_{i \leq k} d_i \leq k(k-1) + \sum_{i>k} \min(d_i, k)$
LGV 引理：DAG 上起點 $a_1..a_n$ 到終點 $b_1..b_n$ 的「兩兩不相交」路徑組數 $= \det(M)$，M_{ij} 為 a_i 到 b_j 的路徑數（前提：只有 $a_i \to b_i$ 的配對可能不相交）。
平面圖 Euler 公式：$V - E + F = 1 + C$（C 為連通塊數，F 含外部面）；簡單平面圖 $V \geq 3$ 時 $E \leq 3V - 6$。

常用恆等式

- Vandermonde：$\sum_k \binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$
- 曲棍球桿：$\sum_{i=r}^{n}\binom{i}{r} = \binom{n+1}{r+1}$
- $\sum i^2 = \frac{n(n+1)(2n+1)}{6}$，$\sum i^3 = \left(\frac{n(n+1)}{2}\right)^2$
- Fibonacci：$\gcd(F_m, F_n) = F_{\gcd(m,n)}$，$F_{m+n} = F_{m+1}F_n + F_m F_{n-1}$

7 Graph

7.1 BCC

```cpp
// 無向圖邊雙連通分量：找出所有橋，並把每個點標上所屬的
// 邊雙分量編號；允許重邊。
// undirected edge-biconnected components; parallel edges
  are allowed
struct edge_bcc {
    struct edge { int to, id; };
    vector<vector<edge>> g;
    vector<int> dfn, low, comp;
    vector<char> bridge;
    int time = 0;

    edge_bcc(int n) : g(n), dfn(n), low(n), comp(n, -1) {}
    void add_edge(int u, int v) {
        int id = bridge.size();
        bridge.push_back(false);
        g[u].push_back({v, id});
        g[v].push_back({u, id});
    }
    void dfs(int u, int parent_edge = -1) {
        dfn[u] = low[u] = ++time;
        for (edge e : g[u]) {
            if (e.id == parent_edge) continue;
            if (!dfn[e.to]) {
                dfs(e.to, e.id);
                low[u] = min(low[u], low[e.to]);
                if (low[e.to] > dfn[u])
                    bridge[e.id] = true;
            } else low[u] = min(low[u], dfn[e.to]);
        }
    }
    void paint(int u, int id) {
        comp[u] = id;
        for (edge e : g[u])
            if (!bridge[e.id] && comp[e.to] == -1)
                paint(e.to, id);
    }
    int solve() {
        for (int u = 0; u < ssize(g); ++u)
            if (!dfn[u]) dfs(u);
        int count = 0;
        for (int u = 0; u < ssize(g); ++u)
            if (comp[u] == -1) paint(u, count++);
        return count;
    }
};
```

7.2 Prim

```cpp
// Prim O(V^2)：完全圖／稠密圖的 MST（稀疏圖用 Kruskal）
// 不存邊，邊權由 w(i, j) 現算；沒邊回傳 T 的最大值
// 回傳 {是否連通, 權重和}
```

```cpp
// 例 1 完全圖（平面點距離）：
// prim<double>(n, [&](int i, int j) {
//     return hypot(x[i] - x[j], y[i] - y[j]); });
// 例 2 鄰接矩陣，g[i][j] == -1 表示沒邊：
// const ll inf = numeric_limits<ll>::max();
// prim<ll>(n, [&](int i, int j) {
//     return g[i][j] == -1 ? inf : g[i][j]; });
template<class T>
pair<bool, T> prim(int n, auto &&w) {
    const T inf = numeric_limits<T>::max();
    vector<T> dist(n, inf);
    vector<char> used(n);
    T total = 0;
    if (n) dist[0] = 0;
    for (int it = 0; it < n; ++it) {
        int u = -1;
        for (int v = 0; v < n; ++v)
            if (!used[v] &&
                (u == -1 || dist[v] < dist[u]))
                u = v;
        if (dist[u] == inf) return {false, total};
        used[u] = true, total += dist[u];
        for (int v = 0; v < n; ++v)
            if (!used[v])
                dist[v] = min(dist[v], w(u, v));
    }
    return {true, total};
}
```

7.3 Bellman Ford

```cpp
// Bellman-Ford 單源最短路，可處理負權邊並偵測從源點可達
// 的負環，O(VE)。
using ll = long long;
const ll inf = LLONG_MAX / 4;

struct edge {
    int from, to;
    ll weight;
};
struct bellman_result {
    vector<ll> dist;
    vector<int> parent;
    bool negative_cycle;
};

bellman_result bellman_ford(int n, int source,
                            const vector<edge> &edges) {
    vector<ll> dist(n, inf);
    vector<int> parent(n, -1);
    dist[source] = 0;
    bool changed = false;
    for (int i = 0; i < n; ++i) {
        changed = false;
        for (edge e : edges) {
            if (dist[e.from] == inf) continue;
            auto x = (__int128)dist[e.from] + e.weight;
            ll nd = clamp<__int128>(x, -inf, inf);
            if (nd < dist[e.to]) {
                dist[e.to] = nd;
                parent[e.to] = e.from;
                changed = true;
            }
        }
        if (!changed) break;
    }
    // reachable negative cycle iff changed
    return {dist, parent, changed};
}

/*
差分約束：x_v - x_u <= w  ->  edge u -> v, weight w
  x_v - x_u >= w -> v -> u, -w；等式拆成兩條
  x_v <= c / x_v >= c：設 x_0 = 0 當成變數處理
  整數嚴格 x_v - x_u < w -> x_v - x_u <= w - 1
加超級源點 s，s -> 每個變數權重 0，從 s 跑 bellman_ford
有負環 -> 無解；否則 x_v = dist[v] (所有 x <= 0 中最大解)
要最小解：約束反向跑最長路
*/
```

7.4 Kruskal

```cpp
// Kruskal 最小生成樹（排序邊 + 並查集），回傳 {是否連通,
// MST 權重和}。
using ll = long long;

struct edge {
    int u, v;
    ll weight;
};
struct dsu {
    vector<int> p, size;
    dsu(int n) : p(n), size(n, 1) {
        iota(p.begin(), p.end(), 0);
    }
    int find(int x) {
        return p[x] == x ? x : p[x] = find(p[x]);
    }
    bool unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) swap(a, b);
        p[b] = a;
        size[a] += size[b];
        return true;
    }
};

pair<bool, ll> kruskal(int n, vector<edge> edges) {
    ranges::sort(edges, {}, &edge::weight);
    dsu uf(n);
    ll total = 0;
    int used = 0;
    for (edge e : edges) if (uf.unite(e.u, e.v))
        total += e.weight, ++used;
    return {used == n - 1, total};
}
```

7.5 Dijkstra

```cpp
// Dijkstra 單源最短路（非負邊權），heap 優化 O(E log V)。
using ll = long long;
using graph_edge = pair<int, ll>; // to, weight
using state = pair<ll, int>;      // distance, vertex
const ll inf = LLONG_MAX / 4;

vector<ll> dijkstra(int source,
                    const vector<vector<graph_edge>> &g) {
    vector<ll> dist(g.size(), inf);
    priority_queue<state, vector<state>, greater<>> pq;
    dist[source] = 0;
    pq.push({0, source}); // distance, vertex
    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d != dist[u]) continue;
        for (auto [v, w] : g[u]) {
            assert(w >= 0);
            if (dist[v] > d + w) {
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

7.6 Hungarian

```cpp
// 二分圖最大匹配（Kuhn 逐點增廣），O(VE)，程式短。
// 可控制增廣順序：左點依權重由大到小 dfs，得到「被配到
// 的左點權重和最大」；也可做字典序最小、逐點加入的增量
// 匹配（HK 是整批增廣，做不到）。
// maximum cardinality bipartite matching
struct bipartite_matching {
    vector<vector<int>> g;
    vector<int> right_match, seen;
    int stamp = 0;
```

```
    bipartite_matching(int left, int right)
        : g(left), right_match(right, -1), seen(right) {}
    void add_edge(int left, int right) {
        g[left].push_back(right);
    }
    bool dfs(int u) {
        for (int v : g[u]) if (seen[v] != stamp) {
            seen[v] = stamp;
            if (right_match[v] == -1 ||
                dfs(right_match[v]))
                return right_match[v] = u, true;
        }
        return false;
    }
    int solve() {
        int ans = 0;
        for (int u = 0; u < ssize(g); ++u)
            ++stamp, ans += dfs(u);
        return ans;
    }
};
```

7.7 Hopcroft-Karp

```
// Hopcroft-Karp 二分圖最大匹配，O(E sqrt(V))，點數邊數
// 大時使用。
// O(E sqrt V); left side [0, n), right side [0, m)
struct hopcroft_karp {
    int n;
    vector<vector<int>> g;
    vector<int> ml, mr, dist; // partner of left / right

    hopcroft_karp(int n, int m)
        : n(n), g(n), ml(n, -1), mr(m, -1), dist(n) {}
    void add_edge(int u, int v) { g[u].push_back(v); }
    bool bfs() {
        queue<int> q;
        bool found = false;
        for (int u = 0; u < n; ++u)
            if (ml[u] == -1) dist[u] = 0, q.push(u);
            else dist[u] = -1;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : g[u]) {
                int w = mr[v];
                if (w == -1) found = true;
                else if (dist[w] == -1)
                    dist[w] = dist[u] + 1, q.push(w);
            }
        }
        return found;
    }
    bool dfs(int u) {
        for (int v : g[u]) {
            int w = mr[v];
            if (~w && dist[w] != dist[u] + 1) continue;
            if (~w && !dfs(w)) continue;
            return ml[u] = v, mr[v] = u, true;
        }
        dist[u] = -1;
        return false;
    }
    int solve() {
        int ans = 0;
        while (bfs())
            for (int u = 0; u < n; ++u)
                if (ml[u] == -1 && dfs(u)) ++ans;
        return ans;
    }
};
```

7.8 KM

```
// KM 演算法：二分圖最大權完美匹配（指派問題），O(n^3)；
// 不存在的邊填 missing。
using ll = long long;
const ll missing = LLONG_MIN / 4;
const ll inf = LLONG_MAX / 4;

struct assignment_result {
    bool perfect;
    ll weight;
    vector<int> right_match;
};

// square maximum-weight assignment; use missing for absent
  edges
assignment_result
max_weight_matching(const vector<vector<ll>> &weight) {
    int n = weight.size();
    for (auto &row : weight) assert(ssize(row) == n);
    vector<ll> u(n + 1), v(n + 1);
    vector<int> p(n + 1), way(n + 1);

    for (int i = 1; i <= n; ++i) {
        p[0] = i;
        vector<ll> best(n + 1, inf);
        vector<char> used(n + 1);
        int col = 0;
        do {
            used[col] = true;
            int row = p[col], next = 0;
            ll delta = inf;
            for (int j = 1; j <= n; ++j) if (!used[j]) {
                ll w = weight[row - 1][j - 1];
                ll cur = (w == missing ? inf / 2 : -w)
                         - u[row] - v[j];
                if (cur < best[j])
                    best[j] = cur, way[j] = col;
                if (best[j] < delta)
                    delta = best[j], next = j;
            }
            for (int j = 0; j <= n; ++j) {
                if (used[j])
                    u[p[j]] += delta, v[j] -= delta;
                else best[j] -= delta;
            }
            col = next;
        } while (p[col]);
        do {
            int prev = way[col];
            p[col] = p[prev];
            col = prev;
        } while (col);
    }

    vector<int> match(n, -1);
    ll sum = 0;
    for (int col = 1; col <= n; ++col) {
        int row = p[col] - 1;
        match[col - 1] = row;
        if (weight[row][col - 1] == missing)
            return {false, 0, {}};
        sum += weight[row][col - 1];
    }
    return {true, sum, match};
}
```

7.9 最小平均環

```
// 最小平均權環（環上邊權平均最小）：Karp 演算法 O(nm)。
using ld = long double;
const ld inf = 1e100L;

struct edge {
    int from, to;
    ld weight;
};

// {has cycle, minimum mean}; karp, O(nm)
pair<bool, ld> min_mean_cycle(int n,
                              const vector<edge> &edges) {
    vector<vector<ld>> dp(n + 1, vector<ld>(n, inf));
    ranges::fill(dp[0], 0); // super source
    for (int k = 1; k <= n; ++k)
        for (edge e : edges) {
            ld from = dp[k - 1][e.from];
            if (from == inf) continue;
```

```
            ld &to = dp[k][e.to];
            to = min(to, from + e.weight);
        }

    ld ans = inf;
    for (int v = 0; v < n; ++v) if (dp[n][v] != inf) {
        ld worst = -inf;
        for (int k = 0; k < n; ++k) if (dp[k][v] != inf) {
            ld d = dp[n][v] - dp[k][v];
            worst = max(worst, d / (n - k));
        }
        ans = min(ans, worst);
    }
    return {ans != inf, ans};
}
```

7.10 floyd+ 負環偵測

```
// Floyd-Warshall 全點對最短路 O(n^3)，並標記哪些點對會
// 受負環影響（距離為 -inf）。
// 單純偵測負環請用 bellman-ford
using ll = long long;
const ll inf = LLONG_MAX / 4;

struct floyd_result {
    vector<vector<ll>> dist;
    vector<vector<char>> negative;
};

floyd_result floyd_warshall(vector<vector<ll>> dist) {
    int n = dist.size();
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i) if (dist[i][k] != inf)
            for (int j = 0; j < n; ++j) {
                if (dist[k][j] == inf) continue;
                auto x = (__int128)dist[i][k] + dist[k][j];
                ll v = clamp<__int128>(x, -inf, inf);
                dist[i][j] = min(dist[i][j], v);
            }

    vector<vector<char>> negative(n, vector<char>(n));
    for (int k = 0; k < n; ++k) if (dist[k][k] < 0)
        for (int i = 0; i < n; ++i) if (dist[i][k] != inf)
            for (int j = 0; j < n; ++j)
                negative[i][j] |= dist[k][j] != inf;
    return {dist, negative};
}
```

7.11 Tarjan

```
// Tarjan 有向圖強連通分量 O(V+E)；註解附無向圖橋與割點
// 的判斷條件。
// 有向圖 SCC，O(V+E)；low[u] == dfn[u] 時彈棧到 u
// dfn：訪問順序；樹邊用 low[v]，棧內已訪問邊用 dfn[v] 更新
  low[u]
// 無向圖橋／割點：見 BCC_edge.cpp；邊雙向加，不用 in_stack
// 只跳過父邊 id（保留重邊）；其餘已訪問邊用 dfn[v] 更新 low[u]
// 橋：DFS 樹邊 (u,v) 在 dfs(v) 後滿足 low[v] > dfn[u]
// 割點：非根 u 有 DFS 子節點 v 滿足 low[v] >= dfn[u]
// DFS 根須有至少兩個 DFS 樹子節點才是割點（不是看度數）
struct tarjan {
    vector<vector<int>> g;
    vector<int> dfn, low, comp, stack;
    vector<char> in_stack;
    int time = 0, count = 0;

    tarjan(int n)
        : g(n), dfn(n), low(n), comp(n, -1), in_stack(n) {}
    void add_edge(int u, int v) { g[u].push_back(v); }
    void dfs(int u) {
        dfn[u] = low[u] = ++time;
        stack.push_back(u);
        in_stack[u] = true;
        for (int v : g[u]) {
            if (!dfn[v])
                dfs(v), low[u] = min(low[u], low[v]);
            else if (in_stack[v])
                low[u] = min(low[u], dfn[v]);
        }
        if (low[u] != dfn[u]) return;
        while (true) {
            int v = stack.back();
            stack.pop_back();
            in_stack[v] = false;
            comp[v] = count;
            if (v == u) break;
        }
        ++count;
    }
    int solve() {
        for (int u = 0; u < ssize(g); ++u)
            if (!dfn[u]) dfs(u);
        return count;
    }
};
```

7.12 Topological Sort

```
// 拓撲排序（Kahn BFS），有環時回傳空陣列，可順便判 DAG。
// empty result means the graph has a cycle
vector<int>
topological_sort(const vector<vector<int>> &g) {
    vector<int> indeg(g.size()), order;
    for (auto &es : g) for (int v : es) ++indeg[v];
    queue<int> q;
    for (int i = 0; i < ssize(g); ++i)
        if (!indeg[i]) q.push(i);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        order.push_back(u);
        for (int v : g[u]) if (!--indeg[v]) q.push(v);
    }
    if (order.size() != g.size()) order.clear();
    return order;
}
```

7.13 LCA

```
// 最近共同祖先：倍增法，O(n log n) 建表、O(log n) 查詢，
// 可算樹上兩點距離。
// binary lifting on a tree; O(n log n) build, O(log n)
  query
struct lca {
    int lg = 1;
    vector<vector<int>> g, up;
    vector<int> depth;

    lca(int n) : g(n), depth(n) {
        while ((1 << lg) < n) ++lg;
        up.assign(lg, vector<int>(n));
    }
    void add_edge(int u, int v) {
        g[u].push_back(v), g[v].push_back(u);
    }
    void build(int root = 0) {
        vector<int> stack{root};
        up[0][root] = root;
        while (!stack.empty()) {
            int u = stack.back();
            stack.pop_back();
            for (int v : g[u]) if (v != up[0][u]) {
                up[0][v] = u, depth[v] = depth[u] + 1;
                stack.push_back(v);
            }
        }
        for (int k = 1; k < lg; ++k)
            for (int v = 0; v < ssize(depth); ++v)
                up[k][v] = up[k - 1][up[k - 1][v]];
    }
    int query(int a, int b) {
        if (depth[a] < depth[b]) swap(a, b);
        for (int k = lg - 1; k >= 0; --k)
            if (depth[a] - (1 << k) >= depth[b])
                a = up[k][a];
        if (a == b) return a;
        for (int k = lg - 1; k >= 0; --k)
            if (up[k][a] != up[k][b])
```

```
                a = up[k][a], b = up[k][b];
        return up[0][a];
    }
    int dist(int a, int b) {
        return depth[a] + depth[b] -
               2 * depth[query(a, b)];
    }
};
```

7.14 樹鏈剖分 HLD

```
// 樹鏈剖分：把樹上路徑／子樹拆成 O(log n) 段連續區間，
// 搭配線段樹做路徑修改與查詢。
// heavy-light decomposition; pos[] indexes a segment tree
// path(u, v) is O(log n) ranges [l, r] of pos
// edge values: store edge (parent, v) at pos[v], skip the
  lca's pos
struct hld {
    vector<vector<int>> g;
    vector<int> parent, depth, heavy, head, pos, size;
    int time = 0;

    hld(int n)
        : g(n), parent(n, -1), depth(n), heavy(n, -1),
          head(n), pos(n), size(n, 1) {}
    void add_edge(int u, int v) {
        g[u].push_back(v), g[v].push_back(u);
    }
    void dfs_size(int u) {
        for (int v : g[u]) if (v != parent[u]) {
            parent[v] = u, depth[v] = depth[u] + 1;
            dfs_size(v);
            size[u] += size[v];
            if (heavy[u] == -1 || size[v] > size[heavy[u]])
                heavy[u] = v;
        }
    }
    void dfs_head(int u, int h) {
        head[u] = h, pos[u] = time++;
        if (heavy[u] != -1) dfs_head(heavy[u], h);
        for (int v : g[u])
            if (v != parent[u] && v != heavy[u])
                dfs_head(v, v);
    }
    void build(int root = 0) {
        dfs_size(root), dfs_head(root, root);
    }
    int lca(int u, int v) {
        for (; head[u] != head[v]; u = parent[head[u]])
            if (depth[head[u]] < depth[head[v]])
                swap(u, v);
        return depth[u] < depth[v] ? u : v;
    }
    vector<pair<int, int>> path(int u, int v) {
        vector<pair<int, int>> res;
        for (; head[u] != head[v]; u = parent[head[u]]) {
            if (depth[head[u]] < depth[head[v]])
                swap(u, v);
            res.push_back({pos[head[u]], pos[u]});
        }
        if (depth[u] > depth[v]) swap(u, v);
        res.push_back({pos[u], pos[v]}); // u is the lca
        return res;
    }
    pair<int, int> subtree(int u) {
        return {pos[u], pos[u] + size[u] - 1};
    }
};
```

7.15 虛樹

```
// 虛樹：只保留關鍵點與它們兩兩的 LCA 建出縮小的樹，多次
// 詢問且關鍵點總數有限時做樹 DP。
// tin: dfs preorder (hld.pos works), lca: e.g. hld.lca
// returns edges (parent, child) of the virtual tree on
// keys and their lcas; the root is the first parent
vector<pair<int, int>> virtual_tree(vector<int> v,
    const vector<int> &tin, auto lca) {
    auto cmp = [&](int a, int b) {
        return tin[a] < tin[b];
    };
    ranges::sort(v, cmp);
    for (int i = 0, k = v.size(); i + 1 < k; ++i)
        v.push_back(lca(v[i], v[i + 1]));
    ranges::sort(v, cmp);
    v.erase(ranges::unique(v).begin(), v.end());
    vector<pair<int, int>> e;
    for (int i = 0; i + 1 < ssize(v); ++i)
        e.push_back({lca(v[i], v[i + 1]), v[i + 1]});
    return e;
}
```

7.16 重心分解

```
// 重心分解：遞迴取重心切樹，深度 O(log n)，用於統計樹上
// 路徑（如距離 <= k 的點對數）。
// centroid decomposition; cpar[] is the parent in the
  centroid tree
// centroid tree depth is O(log n); every path passes some
  centroid
struct centroid {
    vector<vector<int>> g;
    vector<int> size, cpar;
    vector<char> removed;

    centroid(int n)
        : g(n), size(n), cpar(n, -1), removed(n) {}
    void add_edge(int u, int v) {
        g[u].push_back(v), g[v].push_back(u);
    }
    int calc_size(int u, int p) {
        size[u] = 1;
        for (int v : g[u])
            if (v != p && !removed[v])
                size[u] += calc_size(v, u);
        return size[u];
    }
    int find(int u, int p, int total) {
        for (int v : g[u])
            if (v != p && !removed[v] &&
                size[v] * 2 > total)
                return find(v, u, total);
        return u;
    }
    void build(int u = 0, int p = -1) {
        int c = find(u, -1, calc_size(u, -1));
        cpar[c] = p, removed[c] = true;
        // solve(c): count paths through c using vertices
          not removed
        for (int v : g[c]) if (!removed[v]) build(v, c);
    }
};
```

7.17 2-SAT

```
// 2-SAT：每個布林變數二選一、限制為「a 或 b」形式，以
// SCC 判斷可滿足並求出一組解。
// variable x: node 2x means x is true, 2x+1 means x is
  false
struct two_sat {
    int n, time = 0, count = 0;
    vector<vector<int>> g;
    vector<int> dfn, low, comp, stack;

    two_sat(int n) : n(n), g(2 * n) {}
    // (x == a) or (y == b)
    void either(int x, bool a, int y, bool b) {
        int u = 2 * x + !a, v = 2 * y + !b;
        g[u ^ 1].push_back(v);
        g[v ^ 1].push_back(u);
    }
    void implies(int x, bool a, int y, bool b) {
        either(x, !a, y, b);
    }
    void set(int x, bool a) { either(x, a, x, a); }
    void dfs(int u) {
        dfn[u] = low[u] = ++time;
        stack.push_back(u);
```

```
        for (int v : g[u]) {
            if (!dfn[v])
                dfs(v), low[u] = min(low[u], low[v]);
            else if (comp[v] == -1)
                low[u] = min(low[u], dfn[v]);
        }
        if (low[u] != dfn[u]) return;
        for (int v = -1; v != u; stack.pop_back())
            comp[v = stack.back()] = count;
        ++count;
    }
    // nullopt if unsatisfiable
    optional<vector<bool>> solve() {
        dfn.assign(2 * n, 0), low.assign(2 * n, 0);
        comp.assign(2 * n, -1);
        for (int i = 0; i < 2 * n; ++i)
            if (!dfn[i]) dfs(i);
        vector<bool> ans(n);
        for (int i = 0; i < n; ++i) {
            if (comp[2 * i] == comp[2 * i + 1])
                return nullopt;
            // tarjan numbers SCCs in reverse topological
              order
            ans[i] = comp[2 * i] < comp[2 * i + 1];
        }
        return ans;
    }
};
```

7.18 歐拉路徑

```
// Hierholzer 求歐拉路徑／迴路（每條邊恰走一次），有向、
// 無向圖皆可，O(V+E)。
// Hierholzer, O(V+E); works for directed and undirected
  graphs
struct euler {
    int n;
    bool directed;
    vector<vector<pair<int, int>>> g; // to, edge id
    vector<pair<int, int>> edges;

    euler(int n, bool directed)
        : n(n), directed(directed), g(n) {}
    void add_edge(int u, int v) {
        int id = edges.size();
        edges.push_back({u, v});
        g[u].push_back({v, id});
        if (!directed) g[v].push_back({u, id});
    }
    // vertex sequence of an euler path (a circuit if
      possible); empty if none
    vector<int> path() {
        int m = edges.size(), s = m ? edges[0].first : 0,
            odd = 0;
        vector<int> deg(n);
        for (auto [u, v] : edges)
            ++deg[u], directed ? --deg[v] : ++deg[v];
        for (int i = 0; i < n; ++i) {
            int d = directed ? deg[i] : deg[i] % 2;
            if (d == 1) s = i, ++odd;
            else if (d != 0 && !(directed && d == -1))
                return {};
        }
        if (odd > (directed ? 1 : 2)) return {};

        vector<int> it(n), ans, stack{s};
        vector<char> used(m);
        while (!stack.empty()) {
            int u = stack.back(), &i = it[u];
            auto &e = g[u];
            while (i < ssize(e) && used[e[i].second]) ++i;
            if (i == ssize(e)) {
                ans.push_back(u), stack.pop_back();
                continue;
            }
            used[e[i].second] = true;
            stack.push_back(e[i].first);
        }
        // edges are not connected
        if (ssize(ans) != m + 1) return {};
        ranges::reverse(ans);
        return ans;
    }
};
```

7.19 一般圖匹配

```
// 一般圖（非二分圖）最大匹配：Edmonds 開花演算法，
// O(V^3)。
// Edmonds blossom, O(V^3); match[v] is the partner or -1
struct general_matching {
    int n;
    vector<vector<int>> g;
    vector<int> match, parent, base, q;
    vector<char> used, blossom;

    general_matching(int n) : n(n), g(n), match(n, -1) {}
    void add_edge(int u, int v) {
        g[u].push_back(v), g[v].push_back(u);
    }
    int lca(int a, int b) {
        vector<char> seen(n);
        for (;;) {
            a = base[a], seen[a] = true;
            if (match[a] == -1) break;
            a = parent[match[a]];
        }
        for (;; b = parent[match[b]])
            if (b = base[b]; seen[b]) return b;
    }
    void mark_path(int v, int b, int child) {
        while (base[v] != b) {
            int m = match[v];
            blossom[base[v]] = blossom[base[m]] = true;
            parent[v] = child, child = m, v = parent[m];
        }
    }
    void push(int i) { used[i] = true, q.push_back(i); }
    int find_path(int root) {
        used.assign(n, false), parent.assign(n, -1);
        base.resize(n);
        iota(base.begin(), base.end(), 0);
        q.clear(), push(root);
        for (int h = 0; h < ssize(q); ++h) {
            int v = q[h];
            for (int to : g[v]) {
                if (base[v] == base[to] || match[v] == to)
                    continue;
                int m = match[to];
                bool in_tree = m != -1 && parent[m] != -1;
                if (to == root || in_tree) {
                    int cur = lca(v, to);
                    blossom.assign(n, false);
                    mark_path(v, cur, to);
                    mark_path(to, cur, v);
                    for (int i = 0; i < n; ++i)
                        if (blossom[base[i]]) {
                            base[i] = cur;
                            if (!used[i]) push(i);
                        }
                } else if (parent[to] == -1) {
                    parent[to] = v;
                    if (m == -1) return to;
                    push(m);
                }
            }
        }
        return -1;
    }
    int solve() {
        int ans = 0;
        for (int v = 0; v < n; ++v) if (match[v] == -1) {
            int u = find_path(v);
            if (u != -1) ++ans;
            while (u != -1) {
                int pv = parent[u], next = match[pv];
                match[u] = pv, match[pv] = u, u = next;
            }
        }
        return ans;
```

```
    }
};
```

7.20 匹配相關定理

```
// 匹配、點覆蓋、邊覆蓋、獨立集、路徑覆蓋之間的關係式
//  (König 定理等)。
最大匹配 + 最小邊覆蓋 = V
最大獨立集 + 最小點覆蓋 = V
最大匹配 = 最小點覆蓋
```

7.21 Dominator Tree

```
// 支配樹：從起點出發，求每個點的最近必經點 idom（所有路
// 徑都必須經過的點）。
// Lengauer-Tarjan, O((V+E) log V); directed graph g from
  root
// returns idom; idom[root] = root, unreachable vertices get
  -1
vector<int> dominator_tree(const vector<vector<int>> &g,
                           int root) {
    int n = g.size(), m = 0;
    vector<vector<int>> rg(n), bucket(n);
    vector<int> dfn(n, -1), order, par(n), sd(n), id(n),
        anc(n), lab(n);
    for (int u = 0; u < n; ++u)
        for (int v : g[u]) rg[v].push_back(u);
    auto dfs = [&](auto &self, int u) -> void {
        dfn[u] = m++, order.push_back(u);
        for (int v : g[u])
            if (dfn[v] == -1)
                self(self, v), par[dfn[v]] = dfn[u];
    };
    dfs(dfs, root);
    // below every index is a dfn
    iota(sd.begin(), sd.end(), 0), anc = lab = sd;
    auto compress = [&](auto &self, int v) -> void {
        if (anc[anc[v]] == anc[v]) return;
        self(self, anc[v]);
        if (sd[lab[anc[v]]] < sd[lab[v]])
            lab[v] = lab[anc[v]];
        anc[v] = anc[anc[v]];
    };
    auto eval = [&](int v) {
        if (anc[v] == v) return v;
        compress(compress, v);
        return lab[v];
    };
    for (int w = m - 1; w > 0; --w) {
        for (int v : rg[order[w]])
            if (dfn[v] != -1)
                sd[w] = min(sd[w], sd[eval(dfn[v])]);
        bucket[sd[w]].push_back(w);
        anc[w] = par[w];
        for (int v : bucket[par[w]]) {
            int u = eval(v);
            id[v] = sd[u] < sd[v] ? u : par[w];
        }
        bucket[par[w]].clear();
    }
    vector<int> idom(n, -1);
    for (int w = 1; w < m; ++w)
        if (id[w] != sd[w]) id[w] = id[id[w]];
    for (int w = 0; w < m; ++w)
        idom[order[w]] = order[w ? id[w] : 0];
    return idom;
}
```

7.22 Stoer-Wagner 全域最小割

```
// Stoer-Wagner 無向圖全域最小割（不指定源匯點，把圖切成
// 兩半的最小代價），O(V^3)。
// global min cut, undirected, adjacency matrix, O(V^3)
// n >= 2, weights >= 0; parallel edges: add the weights
using ll = long long;

ll stoer_wagner(vector<vector<ll>> w) {
    int n = w.size();
    ll best = LLONG_MAX;
    vector<int> alive(n);
    iota(alive.begin(), alive.end(), 0);
    for (int phase = n; phase > 1; --phase) {
        vector<ll> key(n);
        vector<char> vis(n);
        int prev = -1, last = -1;
        auto val = [&](int v) {
            return vis[v] ? -1 : key[v];
        };
        for (int it = 0; it < phase; ++it) {
            int s = *ranges::max_element(alive, {}, val);
            vis[s] = true, prev = last, last = s;
            for (int v : alive)
                if (!vis[v]) key[v] += w[s][v];
        }
        best = min(best, key[last]); // cut of the phase
        for (int v : alive) { // merge last into prev
            w[prev][v] += w[last][v];
            w[v][prev] = w[prev][v];
        }
        alive.erase(ranges::find(alive, last));
    }
    return best;
}
```

8 Data Structure

8.1 2D Range Tree

```
// 靜態二維區域樹：離線點集，查詢矩形內點的數量，
// O(log^2 n)。
// static rectangle point count, inclusive bounds
struct range_tree_2d {
    vector<int> xs;
    vector<vector<int>> ys;

    range_tree_2d(vector<pair<int, int>> p) {
        ranges::sort(p);
        for (auto q : p) xs.push_back(q.first);
        ys.resize(4 * max<size_t>(1, p.size()));
        if (!p.empty()) build(1, 0, p.size(), p);
    }
    void build(int o, int l, int r,
               const vector<pair<int, int>> &p) {
        if (r - l == 1)
            return ys[o].push_back(p[l].second);
        int m = (l + r) / 2;
        build(o * 2, l, m, p);
        build(o * 2 + 1, m, r, p);
        ranges::merge(ys[o * 2], ys[o * 2 + 1],
                      back_inserter(ys[o]));
    }
    int query(int ql, int qr, int y1, int y2, int o, int l,
              int r) {
        if (ql <= l && r <= qr)
            return ranges::upper_bound(ys[o], y2) -
                   ranges::lower_bound(ys[o], y1);
        int m = (l + r) / 2, ans = 0;
        if (ql < m)
            ans += query(ql, qr, y1, y2, o * 2, l, m);
        if (m < qr)
            ans += query(ql, qr, y1, y2, o * 2 + 1, m, r);
        return ans;
    }
    int count(int x1, int y1, int x2, int y2) {
        int l = ranges::lower_bound(xs, x1) - xs.begin();
        int r = ranges::upper_bound(xs, x2) - xs.begin();
        if (l == r) return 0;
        return query(l, r, y1, y2, 1, 0, xs.size());
    }
};
```

8.2 Segment Tree

```
// 線段樹：區間加值、區間最大值（帶懶標記）。
// range add, range maximum; public ranges are [l, r]
```

```
struct segtree {
    struct node { long long mx = 0, lazy = 0; };
    int n;
    vector<node> t;

    segtree(int n) : n(n), t(4 * max(1, n)) {
        assert(n > 0);
    }
    void apply(int o, long long v) {
        t[o].mx += v, t[o].lazy += v;
    }
    void push(int o) {
        if (!t[o].lazy) return;
        apply(o * 2, t[o].lazy);
        apply(o * 2 + 1, t[o].lazy);
        t[o].lazy = 0;
    }
    void add(int ql, int qr, long long v, int o, int l,
             int r) {
        if (ql <= l && r <= qr) return apply(o, v);
        push(o);
        int m = (l + r) / 2;
        if (ql < m) add(ql, qr, v, o * 2, l, m);
        if (m < qr) add(ql, qr, v, o * 2 + 1, m, r);
        t[o].mx = max(t[o * 2].mx, t[o * 2 + 1].mx);
    }
    long long max_query(int ql, int qr, int o, int l,
                        int r) {
        if (ql <= l && r <= qr) return t[o].mx;
        push(o);
        int m = (l + r) / 2;
        long long ans = LLONG_MIN;
        if (ql < m)
            ans = max(ans, max_query(ql, qr, o * 2, l, m));
        if (m < qr)
            ans = max(ans,
                      max_query(ql, qr, o * 2 + 1, m, r));
        return ans;
    }
    void add(int l, int r, long long v) {
        if (l <= r) add(l, r + 1, v, 1, 0, n);
    }
    long long max_query(int l, int r) {
        assert(l <= r);
        return max_query(l, r + 1, 1, 0, n);
    }
};
```

8.3 ZKW 線段樹

```
// ZKW 非遞迴線段樹：單點加值、區間和，常數小、程式短。
// iterative segment tree: point add and range sum [l, r]
struct zkw_sum {
    int n = 1;
    vector<long long> t;
    zkw_sum(int size) {
        while (n < size) n *= 2;
        t.assign(2 * n, 0);
    }
    void add(int p, long long v) {
        for (t[p += n] += v, p /= 2; p; p /= 2)
            t[p] = t[p * 2] + t[p * 2 + 1];
    }
    long long sum(int l, int r) {
        long long ans = 0;
        for (l += n, r += n + 1; l < r; l /= 2, r /= 2) {
            if (l & 1) ans += t[l++];
            if (r & 1) ans += t[--r];
        }
        return ans;
    }
};
```

8.4 Sparse Table

```
// 稀疏表：靜態陣列 O(n log n) 預處理、O(1) 區間最小值
// (RMQ)。
// idempotent rmq: minimum on [l, r]
template<class T>
struct sparse_table {
    vector<vector<T>> st;

    sparse_table(const vector<T> &a) {
        int k = a.empty() ? 0 : __lg(a.size()) + 1;
        st.assign(k, vector<T>(a.size()));
        if (a.empty()) return;
        st[0] = a;
        for (int j = 1; j < k; ++j)
            for (int i = 0; i + (1 << j) <= ssize(a); ++i)
                st[j][i] =
                    min(st[j - 1][i],
                        st[j - 1][i + (1 << (j - 1))]);
    }
    T query(int l, int r) {
        assert(l <= r);
        int k = __lg(r - l + 1);
        return min(st[k][l], st[k][r - (1 << k) + 1]);
    }
};
```

8.5 Lazy Tag

```
// 懶標記線段樹：區間加值、區間和。
// range add, range sum; public ranges are [l, r]
struct lazy_sum {
    int n;
    vector<long long> sum, tag;
    lazy_sum(int n)
        : n(n), sum(4 * max(1, n)), tag(sum.size()) {
        assert(n > 0);
    }

    void apply(int o, int l, int r, long long v) {
        sum[o] += v * (r - l);
        tag[o] += v;
    }
    void push(int o, int l, int r) {
        if (!tag[o]) return;
        int m = (l + r) / 2;
        apply(o * 2, l, m, tag[o]);
        apply(o * 2 + 1, m, r, tag[o]);
        tag[o] = 0;
    }
    void add(int ql, int qr, long long v, int o, int l,
             int r) {
        if (ql <= l && r <= qr) return apply(o, l, r, v);
        push(o, l, r);
        int m = (l + r) / 2;
        if (ql < m) add(ql, qr, v, o * 2, l, m);
        if (m < qr) add(ql, qr, v, o * 2 + 1, m, r);
        sum[o] = sum[o * 2] + sum[o * 2 + 1];
    }
    long long query(int ql, int qr, int o, int l, int r) {
        if (ql <= l && r <= qr) return sum[o];
        push(o, l, r);
        int m = (l + r) / 2;
        long long ans = 0;
        if (ql < m) ans += query(ql, qr, o * 2, l, m);
        if (m < qr) ans += query(ql, qr, o * 2 + 1, m, r);
        return ans;
    }
    void add(int l, int r, long long v) {
        if (l <= r) add(l, r + 1, v, 1, 0, n);
    }
    long long query(int l, int r) {
        return l <= r ? query(l, r + 1, 1, 0, n) : 0;
    }
};
```

8.6 Lazy Min / 線段樹上二分

```
// 區間加值、區間最小值的懶標記線段樹，並支援線段樹上二
// 分：找區間內第一個／最後一個 <= x 的位置。
// range add, range min, segment tree binary search, n >= 1
// public ranges are [l, r]; first/last_le: -1 if none
using ll = long long;

struct lazy_min {
```

```
    int n, fl, fr; // fl, fr, fx, rev: arguments of find()
    ll fx;
    bool rev;
    vector<ll> mn, tag;
    lazy_min(const vector<ll> &a)
        : n(a.size()), mn(4 * n), tag(4 * n) {
        build(a, 1, 0, n);
    }
    void pull(int o) {
        mn[o] = min(mn[o * 2], mn[o * 2 + 1]);
    }
    void apply(int o, ll v) { mn[o] += v, tag[o] += v; }
    void push(int o) {
        apply(o * 2, tag[o]), apply(o * 2 + 1, tag[o]);
        tag[o] = 0;
    }
    void build(const vector<ll> &a, int o, int l, int r) {
        if (r - l == 1) return void(mn[o] = a[l]);
        int m = (l + r) / 2;
        build(a, o * 2, l, m), build(a, o * 2 + 1, m, r);
        pull(o);
    }
    void add(int ql, int qr, ll v, int o, int l, int r) {
        if (qr <= l || r <= ql) return;
        if (ql <= l && r <= qr) return apply(o, v);
        push(o);
        int m = (l + r) / 2;
        add(ql, qr, v, o * 2, l, m);
        add(ql, qr, v, o * 2 + 1, m, r);
        pull(o);
    }
    ll query(int ql, int qr, int o, int l, int r) {
        if (qr <= l || r <= ql) return LLONG_MAX;
        if (ql <= l && r <= qr) return mn[o];
        push(o);
        int m = (l + r) / 2;
        return min(query(ql, qr, o * 2, l, m),
                   query(ql, qr, o * 2 + 1, m, r));
    }
    // first (rev: last) i in [fl, fr) with a[i] <= fx
    int find(int o, int l, int r) {
        if (fr <= l || r <= fl || mn[o] > fx) return -1;
        if (r - l == 1) return l;
        push(o);
        int m = (l + r) / 2, L = o * 2, R = o * 2 + 1;
        int a = rev ? find(R, m, r) : find(L, l, m);
        if (a != -1) return a;
        return rev ? find(L, l, m) : find(R, m, r);
    }
    void add(int l, int r, ll v) {
        add(l, r + 1, v, 1, 0, n);
    }
    ll query(int l, int r) {
        return query(l, r + 1, 1, 0, n);
    }
    int first_le(int l, int r, ll x) {
        fl = l, fr = r + 1, fx = x, rev = false;
        return find(1, 0, n);
    }
    int last_le(int l, int r, ll x) {
        fl = l, fr = r + 1, fx = x, rev = true;
        return find(1, 0, n);
    }
};
```

8.7 BIT 樹狀樹組

```
// 樹狀數組 (Fenwick)：單點加值、前綴和／區間和，
// O(log n)。
// public indices are 0-based
struct bit {
    using ll = long long;
    vector<ll> t;
    bit(int n) : t(n + 1) {}

    void add(int p, ll x) {
        for (++p; p < ssize(t); p += p & -p) t[p] += x;
    }
    ll prefix(int r) { // [0, r]
        ll ans = 0;
        for (++r; r > 0; r -= r & -r) ans += t[r];
        return ans;
    }
    ll sum(int l, int r) { // [l, r]
        return l > r ? 0
                     : prefix(r) - (l ? prefix(l - 1) : 0);
    }
};
```

8.8 並查集 union find

```
// 並查集：路徑壓縮 + 按大小合併，判斷連通性、維護集合大
// 小。
struct dsu {
    vector<int> parent, size;
    dsu(int n) : parent(n), size(n, 1) {
        iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        if (parent[x] == x) return x;
        return parent[x] = find(parent[x]);
    }
    void unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return;
        if (size[a] < size[b]) swap(a, b);
        parent[b] = a;
        size[a] += size[b];
    }
    bool same(int a, int b) { return find(a) == find(b); }
};
```

8.9 可撤銷並查集

```
// 可撤銷並查集（不做路徑壓縮）：可回滾到之前的狀態，用
// 於離線動態連通性（線段樹分治）。
// union by size, no path compression: find is O(log n)
// t = d.time(); ...unite...; d.rollback(t) undoes them
// offline dynamic connectivity: put each edge on the
// segment tree over time, dfs it and roll back on return
struct rollback_dsu {
    vector<int> p, size;
    vector<pair<int, int>> history; // (root, merged child)

    rollback_dsu(int n) : p(n), size(n, 1) {
        iota(p.begin(), p.end(), 0);
    }
    int find(int x) {
        while (p[x] != x) x = p[x];
        return x;
    }
    bool unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) swap(a, b);
        p[b] = a, size[a] += size[b];
        history.push_back({a, b});
        return true;
    }
    int time() { return history.size(); }
    void rollback(int t) {
        while (ssize(history) > t) {
            auto [a, b] = history.back();
            history.pop_back();
            p[b] = b, size[a] -= size[b];
        }
    }
};
```

8.10 Treap

```
// Treap（split / merge）：可當平衡樹（依 key）或序列
//  （依位置）使用，支援區間操作、插入刪除。
// implicit/key treap primitives; node 0 means null
struct treap {
    struct node {
        int value = 0, priority = 0, size = 0, left = 0,
            right = 0;
```

```
    };
    struct split_result { int left, right; };
    vector<node> t{node{}};
    mt19937 rng{uint32_t(time(0))};

    int make_node(int value) {
        t.push_back({value, int(rng()), 1, 0, 0});
        return t.size() - 1;
    }
    int size(int u) { return u ? t[u].size : 0; }
    void pull(int u) {
        t[u].size = 1 + size(t[u].left) + size(t[u].right);
    }

    split_result split_size(int u, int k) {
        if (!u) return {0, 0};
        if (size(t[u].left) >= k) {
            auto s = split_size(t[u].left, k);
            t[u].left = s.right;
            pull(u);
            return {s.left, u};
        }
        auto s = split_size(t[u].right,
                            k - size(t[u].left) - 1);
        t[u].right = s.left;
        pull(u);
        return {u, s.right};
    }
    split_result split_value(int u, int value) {
        if (!u) return {0, 0};
        if (t[u].value <= value) {
            auto s = split_value(t[u].right, value);
            t[u].right = s.left;
            pull(u);
            return {u, s.right};
        }
        auto s = split_value(t[u].left, value);
        t[u].left = s.right;
        pull(u);
        return {s.left, u};
    }
    int merge(int a, int b) {
        if (!a || !b) return a ? a : b;
        if (t[a].priority < t[b].priority) {
            t[a].right = merge(t[a].right, b);
            pull(a);
            return a;
        }
        t[b].left = merge(a, t[b].left);
        pull(b);
        return b;
    }
};
```

8.11 2D BIT

```
// 二維 BIT：矩形加值、矩形求和。
// 1-based rectangle add and rectangle sum; memory is 32*n*m
  bytes
struct bit2d {
    using ll = long long;
    int n, m;
    array<vector<vector<ll>>, 4> t;

    bit2d(int n, int m) : n(n), m(m) {
        for (auto &a : t)
            a.assign(n + 1, vector<ll>(m + 1));
    }

    void add_one(int x, int y, ll v) {
        for (int i = x; i <= n; i += i & -i)
            for (int j = y; j <= m; j += j & -j) {
                t[0][i][j] += v;
                t[1][i][j] += v * (x - 1);
                t[2][i][j] += v * (y - 1);
                t[3][i][j] += v * (x - 1) * (y - 1);
            }
    }

    ll get(int k, int x, int y) {
        ll ans = 0;
        for (int i = x; i > 0; i -= i & -i)
            for (int j = y; j > 0; j -= j & -j)
                ans += t[k][i][j];
        return ans;
    }

    void add(int x1, int y1, int x2, int y2, ll v) {
        assert(1 <= x1 && x1 <= x2 && x2 <= n);
        assert(1 <= y1 && y1 <= y2 && y2 <= m);
        add_one(x1, y1, v);
        if (y2 < m) add_one(x1, y2 + 1, -v);
        if (x2 < n) add_one(x2 + 1, y1, -v);
        if (x2 < n && y2 < m) add_one(x2 + 1, y2 + 1, v);
    }

    ll prefix(int x, int y) {
        if (x <= 0 || y <= 0) return 0;
        return get(0, x, y) * x * y - get(1, x, y) * y -
               get(2, x, y) * x + get(3, x, y);
    }
    ll sum(int x1, int y1, int x2, int y2) {
        return prefix(x2, y2) - prefix(x1 - 1, y2) -
               prefix(x2, y1 - 1) + prefix(x1 - 1, y1 - 1);
    }
};
```

8.12 持久化線段樹

```
// 持久化線段樹（主席樹）：每次修改產生新版本；經典用途
// 為靜態區間第 k 小。
// persistent segment tree on [0, n): point add, range sum
// every add returns a new root; root 0 is the empty tree
// reserve about (updates * (log n + 2)) nodes if memory is
  tight
using ll = long long;

struct persistent_seg {
    struct node { int l = 0, r = 0; ll sum = 0; };
    int n;
    vector<node> t{node{}};

    persistent_seg(int n) : n(n) {}
    int add(int u, int l, int r, int p, ll v) {
        int id = t.size();
        t.push_back(t[u]);
        t[id].sum += v;
        if (l < r) {
            // C++17: the right side of = runs first
            int m = (l + r) / 2;
            if (p <= m) t[id].l = add(t[id].l, l, m, p, v);
            else t[id].r = add(t[id].r, m + 1, r, p, v);
        }
        return id;
    }
    int add(int root, int p, ll v) {
        return add(root, 0, n - 1, p, v);
    }
    ll sum(int u, int l, int r, int ql, int qr) {
        if (!u || qr < l || r < ql) return 0;
        if (ql <= l && r <= qr) return t[u].sum;
        int m = (l + r) / 2;
        return sum(t[u].l, l, m, ql, qr) +
               sum(t[u].r, m + 1, r, ql, qr);
    }
    ll sum(int root, int ql, int qr) {
        return sum(root, 0, n - 1, ql, qr);
    }
    // k-th (0-based) smallest position of version b minus
      version a
    // e.g. root[i] = add(root[i-1], a[i], 1);
      kth(root[l-1], root[r], k)
    int kth(int a, int b, ll k) {
        int l = 0, r = n - 1;
        while (l < r) {
            int m = (l + r) / 2;
            ll left = t[t[b].l].sum - t[t[a].l].sum;
            if (k < left) a = t[a].l, b = t[b].l, r = m;
            else {
                k -= left;
```

```
                a = t[a].r, b = t[b].r, l = m + 1;
            }
        }
        return l;
    }
};
```

8.13 李超線段樹

```
// 李超線段樹：動態插入直線 kx+b，查詢某個 x 的最大值；
// 常用於斜率優化 DP。
// Li Chao tree on integer x in [lo, hi]: add line kx+b,
  query max
// for min, insert {-k, -b} and negate the answer
using ll = long long;

struct li_chao {
    struct line {
        ll k = 0, b = LLONG_MIN;
        ll operator()(ll x) const { return k * x + b; }
    };
    struct node { line f; int l = -1, r = -1; };
    ll lo, hi;
    vector<node> t{node{}};

    li_chao(ll lo, ll hi) : lo(lo), hi(hi) {}
    void add(line f) {
        int u = 0;
        for (ll l = lo, r = hi;;) {
            ll m = l + (r - l) / 2;
            bool left = f(l) > t[u].f(l),
                 mid = f(m) > t[u].f(m);
            if (mid) swap(t[u].f, f);
            if (l == r || f.b == LLONG_MIN) return;
            bool go_left = left != mid;
            if (go_left) r = m;
            else l = m + 1;
            int c = go_left ? t[u].l : t[u].r;
            if (c == -1) {
                c = t.size(), t.push_back({});
                (go_left ? t[u].l : t[u].r) = c;
            }
            u = c;
        }
    }
    ll query(ll x) {
        ll ans = LLONG_MIN, l = lo, r = hi;
        for (int u = 0; u != -1;) {
            ans = max(ans, t[u].f(x));
            ll m = l + (r - l) / 2;
            if (x <= m) u = t[u].l, r = m;
            else u = t[u].r, l = m + 1;
        }
        return ans;
    }
};
```

9 String

9.1 KMP

```
// KMP：prefix function（失敗函數），O(n+m) 找出 pattern
// 在 text 中所有出現位置。
vector<int> prefix_function(const string &s) {
    vector<int> pi(s.size());
    for (int i = 1; i < ssize(s); ++i) {
        int j = pi[i - 1];
        while (j && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}

vector<int> kmp(const string &text,
                const string &pattern) {
    if (pattern.empty()) return {};
    vector<int> pi = prefix_function(pattern), pos;
    for (int i = 0, j = 0; i < ssize(text); ++i) {
        while (j && text[i] != pattern[j]) j = pi[j - 1];
        if (text[i] == pattern[j]) ++j;
        if (j == ssize(pattern)) {
            pos.push_back(i - j + 1);
            j = pi[j - 1];
        }
    }
    return pos;
}
```

9.2 smallest rotation

```
// 最小表示法：O(n) 求字串字典序最小的循環位移。
// 回傳字典序最小的循環位移，O(n)
// 例如 "baca" -> "abac"；將開頭若干字元搬到尾端
string smallest_rotation(string s) {
    int n = s.size();
    if (!n) return s;
    s += s;
    int i = 0, j = 1;
    while (i < n && j < n) {
        int k = 0;
        while (k < n && s[i + k] == s[j + k]) ++k;
        if (k == n) break;
        if (s[i + k] < s[j + k]) j += k + 1;
        else i += k + 1;
        if (i == j) ++j;
    }
    return s.substr(min(i, j), n);
}
```

9.3 Suffix Array

```
// 後綴陣列 SA（倍增 + radix sort）與 rank、LCP 陣列；用
// 於不同子字串數、最長重複子字串等。
struct suffix_data {
    vector<int> sa, rank, lcp;
};

suffix_data suffix_array(const string &s) {
    int n = s.size();
    if (!n) return {};
    vector<int> sa(n), rank(n), next(n), temp(n),
        count(max(n, 256));

    for (unsigned char c : s) ++count[c];
    partial_sum(count.begin(), count.end(), count.begin());
    for (int i = n - 1; i >= 0; --i)
        sa[--count[(unsigned char)s[i]]] = i;
    int classes = 0;
    for (int i = 0; i < n; ++i) {
        if (i && s[sa[i]] != s[sa[i - 1]]) ++classes;
        rank[sa[i]] = classes;
    }
    ++classes;

    for (int len = 1; len < n && classes < n; len *= 2) {
        int p = 0;
        for (int i = max(0, n - len); i < n; ++i)
            temp[p++] = i;
        for (int x : sa) if (x >= len) temp[p++] = x - len;

        fill(count.begin(), count.begin() + classes, 0);
        for (int x : temp) ++count[rank[x]];
        partial_sum(count.begin(), count.begin() + classes,
                    count.begin());
        for (int i = n - 1; i >= 0; --i)
            sa[--count[rank[temp[i]]]] = temp[i];

        next[sa[0]] = 0;
        int new_classes = 1;
        for (int i = 1; i < n; ++i) {
            int a = sa[i - 1], b = sa[i];
            int ar = a + len < n ? rank[a + len] : -1;
            int br = b + len < n ? rank[b + len] : -1;
            if (rank[a] != rank[b] || ar != br)
                ++new_classes;
            next[b] = new_classes - 1;
```

```
        }
        rank.swap(next);
        classes = new_classes;
    }

    vector<int> lcp(n);
    for (int i = 0, h = 0; i < n; ++i) {
        if (!rank[i]) continue;
        int j = sa[rank[i] - 1];
        while (i + h < n && j + h < n &&
               s[i + h] == s[j + h])
            ++h;
        lcp[rank[i]] = h;
        if (h) --h;
    }
    return {sa, rank, lcp};
}
```

9.4 Z-value

```
// Z function：z[i] 為 s 與 s[i..] 的最長共同前綴長度；
// 另附 Manacher O(n) 求每個中心的最長回文半徑。
vector<int> z_function(const string &s) {
    int n = s.size();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            ++z[i];
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
    return z;
}

// radius of odd palindrome centered at i, including the
  center
vector<int> manacher_odd(const string &s) {
    int n = s.size();
    vector<int> radius(n);
    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k =
            i > r ? 1 : min(radius[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n &&
               s[i - k] == s[i + k])
            ++k;
        radius[i] = k--;
        if (i + k > r) l = i - k, r = i + k;
    }
    return radius;
}
```

9.5 旋轉哈希

```
// 字串滾動哈希（模 2^61-1），O(1) 取任意子字串的 hash
// 來比較是否相等。
using u64 = uint64_t;
using u128 = __uint128_t;
const u64 mod = (1ULL << 61) - 1;

u64 mod_add(u64 a, u64 b) {
    u64 x = a + b;
    return x >= mod ? x - mod : x;
}
u64 mod_mul(u64 a, u64 b) { return u128(a) * b % mod; }

struct rolling_hash {
    u64 base;
    vector<u64> hash, power;

    rolling_hash(const string &s, u64 base = 911382323)
        : base(base) {
        assert(1 < base && base < mod);
        hash.resize(s.size() + 1);
        power.assign(s.size() + 1, 1);
        for (int i = 0; i < ssize(s); ++i) {
            power[i + 1] = mod_mul(power[i], base);
            hash[i + 1] = mod_add(mod_mul(hash[i], base),
                                  (unsigned char)s[i] + 1);
        }
    }
    u64 get(int l, int r) { // [l, r]
        u64 cut = mod_mul(hash[l], power[r - l + 1]);
        u64 h = hash[r + 1];
        return h >= cut ? h - cut : h + mod - cut;
    }
};
```

9.6 後綴自動機

```
// 後綴自動機（SAM）：逐字加入建出接受所有子字串的自動機；
// 可判斷子字串是否出現、算不同子字串數
// sum(len[v]-len[link[v]])。
// link 樹（後綴連結樹）就是反轉字串的後綴樹。
struct sam {
    struct state {
        int len = 0, link = -1;
        array<int, 26> next{};
    };
    vector<state> st{state{}};
    int last = 0;

    void add(char ch) {
        int c = ch - 'a';
        assert(0 <= c && c < 26);
        int cur = st.size(), p = last;
        st.push_back({});
        st[cur].len = st[last].len + 1;
        while (p != -1 && !st[p].next[c])
            st[p].next[c] = cur, p = st[p].link;
        if (p == -1) st[cur].link = 0;
        else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len)
                st[cur].link = q;
            else {
                int clone = st.size();
                st.push_back(st[q]);
                st[clone].len = st[p].len + 1;
                while (p != -1 && st[p].next[c] == q)
                    st[p].next[c] = clone, p = st[p].link;
                st[q].link = st[cur].link = clone;
            }
        }
        last = cur;
    }
};
```

9.7 AC 自動機

```
// AC 自動機：多模式字串匹配，一次掃 text 統計所有
// pattern 的出現次數。
// lowercase only; call build() after all inserts
// after build, next[] is the full automaton and end counts
  suffix matches
struct aho_corasick {
    struct node {
        array<int, 26> next{};
        int fail = 0, end = 0;
    };
    vector<node> t{node{}};
    // bfs order; reverse it for dp on the fail tree
    vector<int> order;

    int insert(const string &s) {
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (!t[u].next[c])
                t[u].next[c] = t.size(), t.push_back({});
            u = t[u].next[c];
        }
        ++t[u].end;
        return u;
    }
    void build() {
        queue<int> q;
        for (int v : t[0].next) if (v) q.push(v);
        while (!q.empty()) {
```

```
            int u = q.front();
            q.pop();
            order.push_back(u);
            t[u].end += t[t[u].fail].end;
            for (int c = 0; c < 26; ++c) {
                int &v = t[u].next[c];
                int f = t[t[u].fail].next[c];
                if (v) t[v].fail = f, q.push(v);
                else v = f;
            }
        }
    }
    // total pattern occurrences
    long long count(const string &text) {
        long long ans = 0;
        for (int u = 0; char ch : text)
            u = t[u].next[ch - 'a'], ans += t[u].end;
        return ans;
    }
};
```

9.8 回文樹

```
// 回文樹 (eertree)：儲存字串中所有不同的回文子字串，並
// 可算出每個回文的出現次數。
// eertree, lowercase; node 0 has len -1, node 1 has len 0
// every other node is a distinct palindrome; call count()
  for occurrences
struct palindromic_tree {
    struct node {
        int len, link = 0, cnt = 0;
        array<int, 26> next{};
    };
    vector<node> t{{-1}, {0}};
    string s;
    int last = 1;

    int get_link(int u) {
        int i = s.size() - 1;
        while (i - t[u].len - 1 < 0 ||
               s[i - t[u].len - 1] != s[i])
            u = t[u].link;
        return u;
    }
    void add(char ch) {
        s += ch;
        int c = ch - 'a', u = get_link(last);
        if (!t[u].next[c]) {
            int v = t.size();
            t.push_back({t[u].len + 2});
            t[v].link = 1;
            if (t[u].len != -1)
                t[v].link = t[get_link(t[u].link)].next[c];
            t[u].next[c] = v;
        }
        last = t[u].next[c];
        ++t[last].cnt;
    }
    void count() { // links point to shorter, earlier nodes
        for (int i = ssize(t) - 1; i > 1; --i)
            t[t[i].link].cnt += t[i].cnt;
    }
};
```

10 Others

10.1 1D/1D dp 優化

```
// 1D/1D DP 決策單調性優化：以單調 stack 二分轉移點，
// O(n^2) -> O(n log n)；範例為 line-breaking 問題。
using ld = long double;

ld int_power(ld x, int p) {
    ld ans = 1;
    for (; p; p /= 2, x *= x) if (p & 1) ans *= x;
    return ans;
}

// original line-breaking transition; returns dp[n]
ld line_break_dp(const vector<int> &length, int limit,
                 int power) {
    struct interval { int l, r, pos; };
    int n = length.size();
    vector<long long> prefix(n + 1);
    vector<ld> dp(n + 1);
    for (int i = 1; i <= n; ++i)
        prefix[i] = prefix[i - 1] + length[i - 1];
    auto cost = [&](int i, int j) {
        ld extra = fabsl(prefix[i] - prefix[j] + i - j -
                         1 - limit);
        return dp[j] + int_power(extra, power);
    };

    vector<interval> stack(n + 2);
    int top = 0, bottom = 0;
    stack[0] = {1, n + 1, 0};
    for (int i = 1; i <= n; ++i) {
        while (i >= stack[bottom].r) ++bottom;
        dp[i] = cost(i, stack[bottom].pos);
        while (top > bottom && i < stack[top].l &&
               cost(stack[top].l, i) <
                   cost(stack[top].l, stack[top].pos)) {
            stack[top - 1].r = stack[top].r;
            --top;
        }
        int l = stack[top].l, r = stack[top].r,
            old = stack[top].pos;
        while (l < r) {
            int m = (l + r) / 2;
            if (cost(m, i) < cost(m, old)) r = m;
            else l = m + 1;
        }
        if (l < stack[top].r) {
            int old_r = stack[top].r;
            stack[top].r = l;
            stack[++top] = {l, old_r, i};
        }
    }
    return dp[n];
}
```

10.2 Theorem - DP optimization

```
// DP 優化：四邊形不等式與決策單調性（以下都是取 min）
1D/1D：dp[j] = min_{i<j} dp[i] + w(i, j)          O(n^2)
2D/1D：dp[i][j] = min_{i<k<=j} dp[i][k-1] + dp[k][j]
                  + w(i, j)，dp[i][i] = 0          O(n^3)
-------------------------------------------------------
四邊形不等式 (QI)：對所有 a <= b <= c <= d
  w(a, c) + w(b, d) <= w(a, d) + w(b, c)
  白話：兩段「交叉」的和 <= 「一大包一小」的和
  常見成立：w(i, j) = f(s[j] - s[i])，f 凸且 s 遞增
  例如 (區間和)^2、| 區間長 - L|^p (p >= 1)
區間包含單調：a <= b <= c <= d 時 w(b, c) <= w(a, d)
不確定時：先寫 O(n^2) 暴力，印出 opt 看是否單調
-------------------------------------------------------
決策單調性：opt[j] = 讓 dp[j] 最小的 i
  w 滿足 QI => opt[j] <= opt[j+1]
1D/1D 優化成 O(n log n)：
(1) 分層 dp_new[j] = min dp_old[i] + w(i, j)：
    分治 solve(l, r, optl, optr)，暴力求 mid 的 opt，
    左半在 [optl, opt]、右半在 [opt, optr] 找
(2) dp 依賴自己（不分層）：deque 存
    「一段 j 的區間 -> 目前最佳決策 i」
    QI 成立：新決策只會贏「後段」，從尾端彈出、
    二分找分界點（見 1D/1D dp 優化模板）
    不等式反向：新決策只會贏「前段」，改用 stack
(3) w(i, j) 可拆成 a[i] * b[j] + c[i] + d[j]：
    改用斜率優化（凸包）或李超線段樹
-------------------------------------------------------
2D/1D Knuth 優化 O(n^3) -> O(n^2)
  條件：w 滿足 QI 且區間包含單調
  則 opt[i][j-1] <= opt[i][j] <= opt[i+1][j]
  區間長度由小到大算，k 只在上面範圍內枚舉
  例：石子合併（相鄰合併）、最佳二元搜尋樹
```

10.3 Stable Marriage

```cpp
// 穩定婚姻問題 (Gale-Shapley)：依偏好求一組不存在私奔對
// 的穩定配對。
// preference[a][rank] = b; order[b][a] = rank
vector<int>
stable_marriage(const vector<vector<int>> &preference,
                const vector<vector<int>> &order) {
    int n = preference.size();
    vector<int> next(n), partner(n, -1);
    queue<int> free;
    for (int a = 0; a < n; ++a) free.push(a);
    while (!free.empty()) {
        int a = free.front();
        free.pop();
        int b = preference[a][next[a]++];
        if (partner[b] == -1) partner[b] = a;
        else if (order[b][a] < order[b][partner[b]])
            free.push(partner[b]), partner[b] = a;
        else free.push(a);
    }
    return partner; // partner[b] = a
}
```

10.4 莫隊

```cpp
// 莫隊演算法：離線區間查詢，排序後移動左右端點，
// O((n+q) sqrt(n))。
// all ranges are [l, r]
struct query {
    int l, r, id;
};

void sort_mo(vector<query> &queries, int n) {
    int block = max(
        1, int(n / sqrt(max<size_t>(1, queries.size()))));
    ranges::sort(queries, [&](query a, query b) {
        int x = a.l / block, y = b.l / block;
        if (x != y) return x < y;
        return x & 1 ? a.r < b.r : a.r > b.r;
    });
}

/*
start with l=0, r=-1 (empty).
move in this order:
while (l > q.l) add(--l); while (r < q.r) add(++r);
while (l < q.l) remove(l++); while (r > q.r) remove(r--);
*/
```

10.5 莫隊帶修改

```cpp
// 帶修改莫隊：多加時間維度處理單點修改，約 O(n^(5/3))。
// all ranges are [l, r]; time = number of applied updates
struct time_query {
    int l, r, time, id;
};

void sort_mo_with_updates(vector<time_query> &queries,
                          int n) {
    int block = max(1, int(pow(max(1, n), 2.0 / 3)));
    ranges::sort(queries, [&](time_query a, time_query b) {
        int al = a.l / block, bl = b.l / block;
        if (al != bl) return al < bl;
        int ar = a.r / block, br = b.r / block;
        if (ar != br) return al & 1 ? ar < br : ar > br;
        return ar & 1 ? a.time < b.time : a.time > b.time;
    });
}

// start with l=0, r=-1, time=0; apply/undo updates while
  moving time
```

10.6 矩陣乘法

```cpp
// 模 mod 的矩陣乘法 O(n^3)；搭配快速冪可求線性遞迴（如
// 費氏數列）第 n 項。
using ll = long long;
using matrix = vector<vector<ll>>;

matrix multiply(const matrix &a, const matrix &b, ll mod) {
    assert(!a.empty() && !b.empty() &&
           a[0].size() == b.size());
    matrix c(a.size(), vector<ll>(b[0].size()));
    for (int i = 0; i < ssize(a); ++i)
        for (int k = 0; k < ssize(b); ++k)
            for (int j = 0; j < ssize(b[0]); ++j) {
                ll x = (__int128)a[i][k] * b[k][j] % mod;
                c[i][j] = (c[i][j] + x) % mod;
            }
    return c;
}
```

10.7 c++ 小抄

```cpp
// C++ 小技巧：pb_ds 的 ordered set（第 k 小／ rank）與可
// 合併 heap、__builtin 位元函數、亂數、去重。
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <ext/pb_ds/priority_queue.hpp>
using namespace __gnu_pbds;

using ordered_set =
    tree<int, null_type, less<int>, rb_tree_tag,
         tree_order_statistics_node_update>;
// s.find_by_order(k): iterator to k-th smallest, 0-based
// s.order_of_key(x): number of values strictly smaller than
  x

using pairing_heap =
    __gnu_pbds::priority_queue<int, less<int>,
                               pairing_heap_tag>;
// auto it = q.push(x); q.modify(it, y); q.erase(it);
  q.join(other);

// __builtin_popcount(x), __builtin_popcountll(x)
// __builtin_clz(x), __builtin_ctz(x): x must be nonzero
// __builtin_*_overflow(a, b, &answer)

int random_int(int l, int r) {
    static mt19937 rng(chrono::steady_clock::now()
                           .time_since_epoch()
                           .count());
    return uniform_int_distribution<int>(l, r)(rng);
}

// sorted unique:
// a.erase(ranges::unique(a).begin(), a.end());
```

10.8 python 小抄

```python
#!/usr/bin/env python3
# Python 小抄：快速讀整數、LCS 範例、stack / queue / 去
# 重寫法。
from collections import deque
import sys


def ints():
    return map(int, input().split())


def all_ints():
    return list(map(int, sys.stdin.buffer.read().split()))


def lcs(a, b):
    dp = [0] * (len(b) + 1)
    for x in a:
        prev = 0
        for j, y in enumerate(b, 1):
            old = dp[j]
            if x == y:
                dp[j] = prev + 1
            else:
                dp[j] = max(dp[j], dp[j - 1])
            prev = old
```

```
    return dp[-1]


# stack: append, pop, stack[-1]
# queue = deque(); append, popleft, queue[0]
# sorted unique values: sorted(set(values))
```

10.9 萬年曆

Zeller 公式：由年月日計算星期幾。

$$h = \left(q + \left\lfloor \frac{13(m+1)}{5} \right\rfloor + K + \left\lfloor \frac{K}{4} \right\rfloor + \left\lfloor \frac{J}{4} \right\rfloor + 5J\right) \bmod 7$$

h： 星期（0 = 星期六, 1 = 星期日, 2 = 星期一, …）
q： 日期（日）
m： 月份（3= 三月, …；一、二月改成 13、14，且年份先減一）
K： 調整後年份的後兩位數（year mod 100）
J： 調整後年份的前兩位數（year ÷ 100）

10.10 多重背包

```
// 多重背包（每種物品有數量上限），求總重不超過 w 的最大
// 價值。
using ll = long long;

struct item {
    int weight, count;
    ll value;
};

// dp[w] = maximum value with total weight at most w
vector<ll> bounded_knapsack(int capacity,
                            const vector<item> &items) {
    vector<ll> dp(capacity + 1);
    for (item x : items) {
        assert(x.weight >= 0 && x.count >= 0);
        if (!x.weight) {
            if (x.value > 0)
                for (ll &value : dp)
                    value += x.value * x.count;
            continue;
        }
        for (int r = 0; r < x.weight && r <= capacity;
             ++r) {
            deque<pair<ll, int>> q;
            for (int k = 0, w = r; w <= capacity;
                 ++k, w += x.weight) {
                ll value = dp[w] - ll(k) * x.value;
                while (!q.empty() &&
                       q.back().first <= value)
                    q.pop_back();
                q.push_back({value, k});
                while (q.front().second < k - x.count)
                    q.pop_front();
                dp[w] = q.front().first + ll(k) * x.value;
            }
        }
    }
    return dp;
}
```

10.11 LIS

```
// 最長嚴格遞增子序列 O(n log n)，回傳子序列的索引；要非
// 嚴格遞增改用 upper_bound。
// longest strictly increasing subsequence, O(n log n)
// non-decreasing: use upper_bound; returns indices
vector<int> lis(const vector<long long> &a) {
    vector<long long> tail;
    vector<int> id, prev(a.size(), -1);
    for (int i = 0; i < ssize(a); ++i) {
        int k =
            ranges::lower_bound(tail, a[i]) - tail.begin();
        if (k == ssize(tail))
            tail.push_back(a[i]), id.push_back(i);
        else tail[k] = a[i], id[k] = i;
        if (k) prev[i] = id[k - 1];
    }
    vector<int> res;
    for (int i = id.empty() ? -1 : id.back(); i != -1;
         i = prev[i])
        res.push_back(i);
    ranges::reverse(res);
    return res;
}
```